

Bases de Datos para inteligencia Artificial

Trabajo práctico

Alumnos:

Lourdes Gonzalez Branchi

Ronald Uthurralt

Juan Pablo Skobalski

Luis David Díaz Charris

1. Descripción del caso de uso

Para el desarrollo del trabajo práctico, el grupo seleccionó el caso de uso No. 8 propuesto por el profesor, titulado “*Sistema de detección de fraude o anomalías*”. A continuación, se realiza el análisis del caso de uso seleccionado.

1.1. Descripción del problema y contexto

En las plataformas transaccionales modernas, como aplicaciones de pago o e-commerce, el volumen y la velocidad de las operaciones hacen imposible la supervisión humana directa en tiempo real. Esto expone a la organización a pérdidas financieras por fraude, ataques automatizados y suplantación de identidad.

El contexto de aplicación abarca un entorno donde coexisten dos necesidades críticas: por un lado, detectar comportamientos inusuales mientras el usuario ejecuta la transacción para autorizarla o bloquearla sin afectar la experiencia de uso. Por otro lado, gestionar el ciclo de vida completo de la investigación de fraudes a través de la generación de alertas, revisiones manuales, documentación de evidencia y la trazabilidad legal.

1.2. ¿Qué problema busca resolver la solución?

La solución busca resolver la vulnerabilidad ante el fraude operativo y la falta de trazabilidad en las decisiones de riesgo, los cuales se podrían concretar en:

- a) La detección tardía de anomalías e intentos de fraude.
- b) La falta de centralización entre la evaluación automatizada y el trabajo de los analistas humanos.
- c) El riesgo de opacidad en las auditorías, relacionado con las razones de rechazo o aprobación de las transacciones, las reglas o modelos de Machine Learning para la evaluación y el o los responsables de la decisión final.

1.3. Usuarios principales y roles del sistema

Se considera que el sistema debería contar con un control de acceso basado en roles (RBAC) para mantener la separación de funciones. A cada usuario le corresponderá un rol y una función específica. Sin embargo, para el alcance de este trabajo no se llega a implementar

completamente, y sólo se llega hasta la definición de los roles para el registro de algunos datos de ejemplo, que servirán para realizar consultas SQL. La implementación completa del RBAC queda como trabajo futuro. A continuación, se definen los roles, permisos y funciones.

Tabla 1. Usuarios/roles y funciones.

Usuario/Rol	Permisos	Función
Analistas de fraude / riesgo	Consulta de casos asignados y en cola. Visualización de evidencia (datos de transacción, dispositivo, ubicación, acumulados históricos). Toma de decisiones de aprobación, rechazo o escalamiento sobre casos asignados. Adjunto de documentación o evidencia al caso. Restricción: sin acceso a configuración de reglas, umbrales de scoring, parámetros de modelos ni gestión de usuarios.	Investigan las alertas generadas por el sistema, evalúan la evidencia (dispositivo, ubicación, montos) y determinan si una operación es legítima o fraudulenta.
Supervisores	Todos los permisos del Analista de Fraude. Reapertura de casos cerrados (cuando se requiere revisión adicional). Autorización de decisiones de alto impacto (ej. aprobación de montos que superan un umbral crítico, escalamiento a instancias externas). Gestión de carga de trabajo: asignación, reasignación y balanceo de casos entre operadores y analistas. Calibración de reglas: propuesta y ajuste de parámetros y umbrales (con aprobación del Administrador). Visión global del desempeño del equipo (métricas de resolución, tiempos de respuesta). Restricción: sin acceso directo a la configuración final de modelos de ML ni a la gestión de usuarios y roles.	Aprueban decisiones de alto impacto (ej. Reabrir un caso, autorizar montos elevados), gestionan la carga de trabajo de los operadores y calibran las reglas del sistema.
Audidores (internos / externos)	Permisos de solo lectura sobre todos los logs y registros de auditoría. Consulta del historial completo de acciones de cualquier usuario (ej. quién, qué, cuándo, dónde, cómo). Visualización de casos, decisiones, evidencias y cambios de configuración (sin posibilidad de modificar). Acceso a reportes de cumplimiento y dashboards predefinidos. Restricción: sin capacidad de modificar, crear o eliminar ningún dato del sistema. Sin acceso a datos sensibles de clientes (ej. números de tarjeta completos, datos biométricos).	Consultan el historial de evidencias, decisiones y registros de actividad para verificar el cumplimiento normativo (compliance) y regulatorio.
Administrador de reglas	Creación, modificación y desactivación de reglas de validación (ej. límites de monto, listas negras, patrones de frecuencia).	Configuran y mantienen la lógica de negocio del sistema de evaluación. Son

	Ajuste de umbrales de scoring (ej. puntos de corte para aprobación, rechazo o revisión manual). Gestión de listas negras (usuarios, dispositivos, IPs, países). Visualización del registro de modelos (sin capacidad de promocionar modelos). Restricción: sin acceso a casos de revisión, sin acceso a datos sensibles de clientes, sin capacidad de gestionar usuarios o roles.	responsables de que las reglas reflejen la política de riesgos de la organización y de que los umbrales se ajusten a la evolución del negocio y del fraude.
Administrador de sistema	Control total sobre usuarios, roles y permisos (gestión del RBAC). Configuración de parámetros globales del sistema (ej. tiempos de sesión, políticas de seguridad). Gestión del registro de modelos: promoción de modelos a producción, rollback a versiones anteriores. Configuración de conectores a sistemas externos (procesadores de pago, fuentes de datos). Monitoreo de la salud del sistema (métricas de infraestructura). Restricción: sin acceso a casos de revisión, sin capacidad de modificar reglas de negocio o umbrales (salvo que sea estrictamente necesario por incidencias técnicas).	Gestionan la infraestructura tecnológica, los accesos al sistema y las configuraciones globales. Garantizan la disponibilidad, seguridad y correcto funcionamiento del ecosistema, así como el ciclo de vida de los modelos de ML.

1.4. Procesos y funcionalidades que debe soportar

El sistema se compone de dos grandes flujos de trabajo que operan de manera paralela y complementaria: un flujo online (síncrono), responsable de la evaluación en tiempo real de cada transacción, y un flujo batch (asíncrono), encargado del entrenamiento de modelos, la generación de reportes y la alimentación de dashboards de monitoreo.

La arquitectura completa del sistema, según el caso de uso, es la que se muestra en la Figura 1.

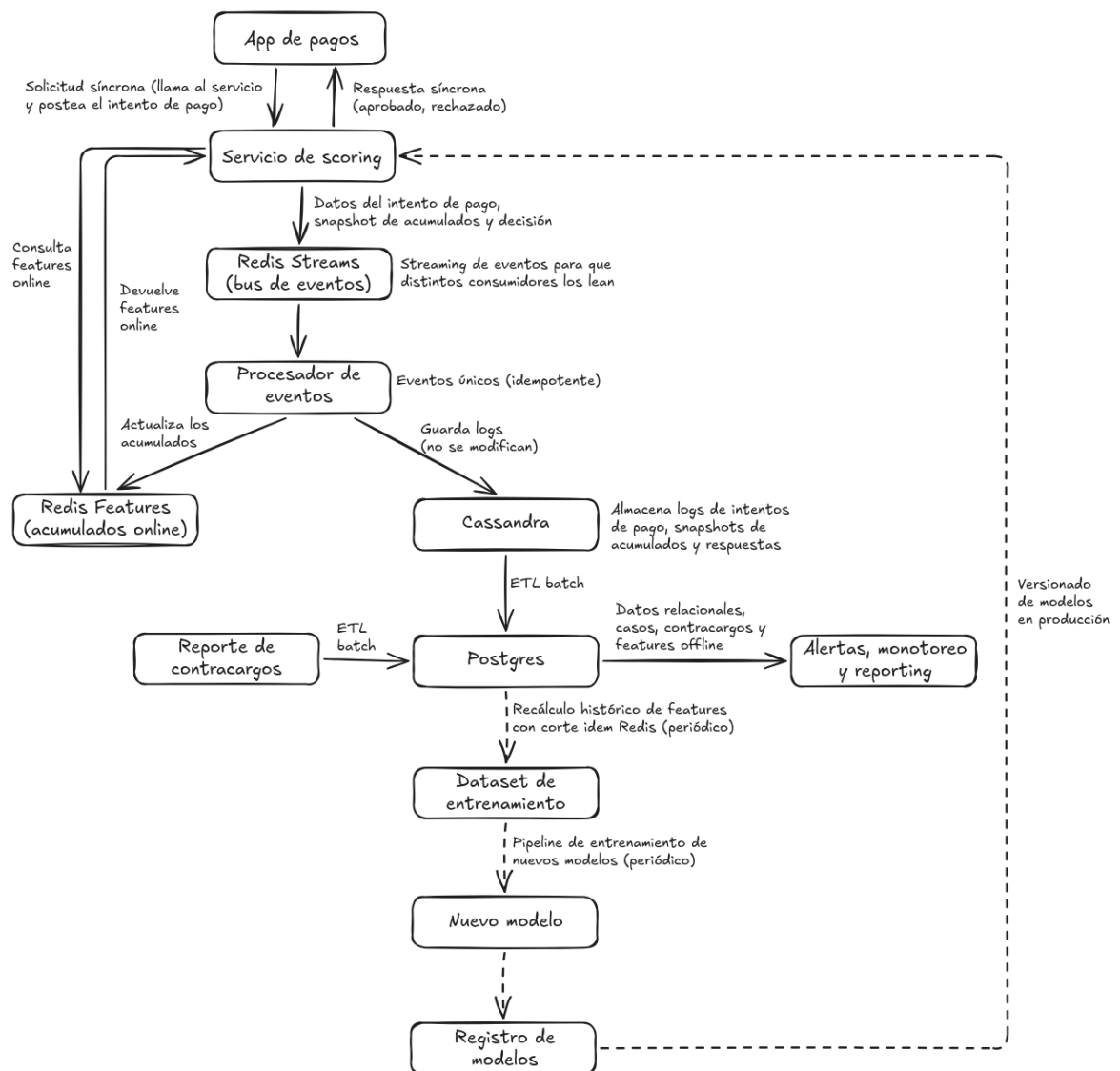


Figura 1. Arquitectura del sistema.

1.4.1. Descripción del flujo online

El flujo online es una etapa del sistema que debe responder en tiempo real para no afectar la experiencia del usuario. Esta etapa tiene tres funciones principales que son:

a) Solicitud de evaluación y consulta de acumulados

El proceso comienza cuando la aplicación de pagos envía una solicitud síncrona al servicio de scoring. Esta solicitud incluye los datos del intento de pago (monto, origen, destino, etc.). Paralelamente, el servicio de scoring consulta los acumulados online almacenados en Redis Features, que contienen métricas históricas del usuario (ej. frecuencia de transacciones, monto promedio, comportamiento reciente). Estos acumulados se actualizan en tiempo real y permiten contextualizar la operación.

b) Evaluación y decisión instantánea

Con la información del pago y los acumulados, el servicio de scoring aplica reglas y modelos cuantitativos para calcular una puntuación de riesgo. En un corto período, el sistema retorna una respuesta síncrona a la aplicación de pagos, que puede ser "aprobado" o "rechazado". Esta decisión se entrega de forma inmediata para no afectar la experiencia del usuario.

c) Publicación de eventos y procesamiento asíncrono

Una vez tomada la decisión, la aplicación de pagos publica los datos completos del intento, incluyendo los features utilizados y la decisión final, en Redis Streams, un bus de eventos que similar a una cola de mensajería. De aquí, un procesador de eventos consume los mensajes de forma asíncrona con dos finalidades: primero, actualizar los acumulados en Redis Features, para que la próxima consulta refleje el nuevo comportamiento; y segundo, guardar logs inmutables en Cassandra, donde se almacenan todos los intentos de pago, los snapshots de los acumulados en el momento de la evaluación y las respuestas emitidas.

1.4.2. Descripción del flujo batch

Este flujo corresponde a la etapa de análisis detallado (normativo) de las transacciones y, además, del entrenamiento de los modelos de inteligencia artificial que servirán para actualizar el servicio de scoring. Esta etapa es más lenta, pero detallada, y es la base de la seguridad del sistema. A continuación, se detallan sus funciones:

a) Alimentación del almacén de datos y reportes

De forma periódica, un proceso ETL batch extrae los datos crudos desde Cassandra y los transforma para cargarlos en Postgres. En esta base de datos relacional se construyen las capas de alertas, monitoreo y reporting, que permiten a los equipos de negocio y riesgos visualizar métricas agregadas, generar alertas automatizadas sobre tendencias anómalas y elaborar dashboards ejecutivos.

b) Integración con contracargos y reentrenamiento de modelos

Paralelamente, el sistema incorpora una fuente externa de reportes de contracargos (chargebacks) provenientes de los procesadores de pago. Estos reportes se cruzan con los datos almacenados en Cassandra mediante otro proceso ETL batch, con dos objetivos principales: uno, recalcular features acumuladas a partir de la relación entre pagos y contracargos, para mejorar la calidad de las variables predictoras en futuras versiones del modelo. Dos, generar un dataset de entrenamiento etiquetado que alimenta un pipeline de entrenamiento periódico. Este pipeline ejecuta automáticamente el ciclo de entrenamiento, validación y selección de nuevos modelos.

c) Registro y despliegue de modelos

Cuando un nuevo modelo supera las métricas de desempeño requeridas, es ingresado en el registro de modelos. Desde allí, puede ser promovido a producción y pasar a ser utilizado por el servicio de scoring en el flujo online, lo que cierra el ciclo de mejora continua.

1.5. Información que se necesita gestionar

El sistema debe administrar un modelo de datos integral que abarque toda la información relevante para la evaluación, el seguimiento y la auditoría de las transacciones. Esta información se organiza en cinco grandes bloques conceptuales, cada uno de los cuales cumple un rol específico dentro del flujo de trabajo descrito anteriormente. Estos bloques son:

Bloque 1: datos de contexto

Este bloque agrupa toda la información que permite caracterizar y contextualizar a la persona o entidad que origina la transacción. Su correcta gestión es fundamental para aplicar políticas de conocimiento del cliente (KYC) y detectar comportamientos anómalos. Incluye:

- **Datos del usuario** (pagador). Identificadores únicos (pagador_id, tipo_identificacion, identificacion_hash), país, estado operativo y fecha de alta.
- **Datos del dispositivo** (dispositivo y pagador_dispositivo). Identificador del dispositivo (dispositivo_id), huella digital (fingerprint), sistema operativo, tipo de dispositivo y marcas temporales de primera y última observación vinculadas al pagador.
- **Ubicación geográfica**. País de origen registrado directamente en el intento de pago (pais_origen).
- **Histórico de comportamiento**. (feature_riesgo / redis features): Métricas acumuladas en Redis Features (ej. acumulados de volumen o frecuencia definidos por ventana temporal) que se actualizan en tiempo real tras cada operación..

Estos datos se utilizan en la evaluación instantánea y alimentan los procesos batch para recalcular features y enriquecer los datasets de entrenamiento.

Bloque 2: datos transaccionales

Este bloque contiene la información específica de cada intento de pago o transacción financiera. Es la materia prima que ingresa al sistema y sobre la cual se aplican las reglas y modelos de scoring. Incluye:

- **Identificación de la operación** (intento_pago). Identificador único de transacción (intento_id) e identificador del evento de origen (evento_id).
- **Datos económicos y temporales**. Monto de la transacción (monto), moneda (moneda) y fecha/hora exacta del intento de pago (fecha_hora).
- **Datos del medio de pago**. Identificador de la tarjeta (tarjeta_id), token de tarjeta (pan_token), marca, país emisor y estado.
- **Información del comercio**. Identificador del cobrador (cobrador_id), nombre del comercio (nom_comercio), categoría, país y estado.

Todos estos datos son recibidos por el servicio de scoring a través de la solicitud síncrona, y posteriormente se almacenan de forma inmutable.

Bloque 3: datos de evaluación

Este bloque engloba toda la información generada durante el proceso de evaluación y que permite entender cómo se llegó a una decisión concreta. Es la "memoria del juicio" del sistema y resulta crítica para auditorías, depuración de errores y mejora continua. Incluye:

- **Puntaje de riesgo emitido (scoring)**. Registro de la evaluación que incluye el identificador de decisión (decision_id), el score_riesgo calculado, la decisión final (ej. aprobado, rechazado), el motivo principal, el monto evaluado y el tiempo de latencia de la evaluación (latencia_ms).
- **Versión del modelo de ML utilizada**. Identificador exacto del modelo de Machine Learning que participó en la evaluación (modelo_version_id), asociado al dataset de entrenamiento correspondiente.
- **Features o variables de entrada**. Mapeo de los valores exactos de las variables de riesgo (feature_id, valor, fecha_calculo_feature) consultadas desde Redis Features que alimentaron la decisión.

- **Reglas de validación aplicadas.** Registro de las reglas de negocio evaluadas (regla_version_id), indicando si la regla fue activada, el valor observado que disparó la regla y el resultado emitido.

Bloque 4: datos de gestión y control

Este bloque cubre toda la información derivada de la interacción humana con el sistema, así como los mecanismos de control y seguimiento que permiten gestionar los casos excepcionales. Es el puente entre la automatización y la supervisión analítica:

- **Monitoreo y reporting de decisiones.** Consultas analíticas en Postgres sobre la tabla DECISION_SCORING y sus reglas/features asociadas para la generación de reportes operativos y análisis de métricas.
- **Configuración de reglas de negocio.** Registro de las reglas definidas por el analista, incluyendo la expresión de condición (expresion_condicion), código de regla, acción asociada, estado y período de vigencia (vigencia_desde, vigencia_hasta).
- **Historial de auditoría por rol.** Registro inmutable de las acciones ejecutadas por los usuarios del sistema (analista_id). Guarda el identificador único del evento (evento_auditoria_id), fecha y hora (fecha_hora), acción realizada (accion), tipo y ID del objeto afectado (tipo_objeto, objeto_id), valores anteriores y nuevos (estado_anterior, estado_nuevo) y el motivo de la acción.

Bloque 5: datos de contracargo

Este bloque de datos es crítico tanto para el **etiquetado supervisado de los modelos** (variable objetivo) como para el **análisis de desempeño** y la **detección de causas raíz**. Los datos se obtienen típicamente a través de los reportes de contracargos de los procesadores de pago (ej. Mastercard GMAP, Visa VROL, o feeds directos de adquirentes). Su correcta gestión permite no solo entrenar modelos más precisos, sino también identificar patrones de fraude recurrente, optimizar estrategias de representment y cumplir con los umbrales regulatorios exigidos por las redes de tarjetas.

Incluye:

- **Identificadores del contracargo.** Identificador único del contracargo (contracargo_id), referencia al intento de pago original (intento_id) y fecha del reporte (fecha_reporte).
- **Datos económicos y causalidad.** Monto disputado (monto), motivo del contracargo (motivo), estado del flujo (estado) y fuente de la información (fuente).
- **Enriquecimiento para entrenamiento de ML.** Integración del contracargo con la decisión de scoring original para generar la etiqueta binaria de fraude (etiqueta_fraude), la fuente de la etiqueta, la fecha de maduración y el identificador de la versión del dataset (dataset_version_id) que alimentará los pipelines de entrenamiento.
- **Reportabilidad de contracargos.** Consolidación en Postgres de los contracargos asociados a reportes periódicos (reporte_id) para el monitoreo de métricas y tasas de disputa.

Estos datos se integran en el pipeline batch de entrenamiento para generar nuevos modelos y alimentan los dashboards de reporting en Postgres, para que los equipos de riesgos y negocio tomen decisiones estratégicas basadas en evidencia sólida y actualizada.

1.6. Riesgos relacionados con los datos

A continuación, se presenta una matriz de riesgos relacionados con los datos.

Tabla 2. Matriz de riesgos relacionados con los datos.

Riesgo de Datos	Impacto	Estrategia de Mitigación / Diseño
Inconsistencia / pérdida por alto volumen	Carga masiva de eventos pierde trazabilidad.	Uso de arquitecturas de event-streaming e inmutabilidad de logs.
Falta de trazabilidad	No saber quién aprobó un fraude o modificó una regla.	Log de auditoría estricto de solo lectura para acciones de usuarios.
Exposición de datos sensibles	Filtración de datos personales, financieros o de ubicación.	Encriptación en reposo/tránsito y enmascaramiento de datos según el rol.
Falsos positivos y bloqueos indebidos	Mala experiencia del usuario legítimo.	Registro detallado de features y decisiones para retroalimentar el pipeline de Machine Learning.

1.7. Decisiones de diseño para el caso

La arquitectura del sistema de gestión de riesgos se fundamenta en un conjunto de decisiones de diseño estratégicas que responden a los requisitos de rendimiento, escalabilidad, trazabilidad y evolución continua del modelo de detección. Estas decisiones han sido adoptadas considerando los volúmenes esperados de transacciones, la necesidad de latencias ultrabajas en la evaluación y la importancia crítica de la auditoría y el cumplimiento normativo.

A continuación, se describen las principales decisiones de diseño que guían la implementación del sistema:

1.7.1. Estrategia de persistencia políglota

El sistema no depende de un único motor de base de datos, sino que emplea diferentes tecnologías de almacenamiento, cada una optimizada para un propósito específico. Esta estrategia permite aprovechar las fortalezas de cada solución y evitar los cuellos de botella que surgirían al forzar una única base de datos a cumplir todos los roles.

- **Bases de datos en memoria.** Se utiliza Redis en dos roles diferenciados:
 - **Redis Features:** es el base de datos basada en el modelo clave-valor y en memoria, que mantiene los acumulados y features de cada entidad actualizados en tiempo real. Debido a que presenta una baja latencia, es útil para que el servicio de scoring consulte estos datos durante la evaluación síncrona sin afectar el tiempo de respuesta de la operación.
 - **Redis Streams:** funciona como un bus de eventos que desacopla el procesamiento en tiempo real de las tareas asíncronas. Los eventos publicados aquí son consumidos por el procesador para actualizar acumulados y persistir logs sin impactar la latencia de la respuesta al cliente.

- **Bases de datos NoSQL basadas en columnas (Cassandra).** Se emplea **Cassandra** como el almacén principal de logs inmutables. Presenta una arquitectura distribuida y un modelo de escritura optimizado (append-only), lo cual permite soportar la ingesta masiva de millones de transacciones sin cuellos de botella. Sus principales funciones son:
 - Almacenar el historial completo de intentos de pago, snapshots de acumulados y decisiones emitidas.
 - Proporcionar una fuente de datos fiable y de alta disponibilidad para los procesos ETL batch que alimentan el reporting y el entrenamiento de modelos.
 - Garantizar la inmutabilidad de los registros, aspecto crítico para la auditoría forense y el cumplimiento normativo.
- **Base de datos relacional (PostgreSQL).** Se utiliza **Postgres** como el sistema de registro para los datos que requieren consistencia transaccional y relaciones complejas entre entidades. Sus funciones son:
 - Gestión de casos, usuarios y roles del sistema.
 - Configuración de reglas de negocio y parámetros de umbrales.
 - Registros de auditoría estructurados que requieren consultas complejas y joins entre tablas.
 - Dashboards de reporting y monitoreo, donde la integridad referencial y la capacidad de realizar consultas analíticas son esenciales.

1.7.2. Arquitectura orientada a eventos

El sistema adopta un modelo de arquitectura orientada a eventos que separa de manera clara el procesamiento síncrono (en línea) del procesamiento asíncrono (en segundo plano). Esta decisión es fundamental para garantizar que la experiencia del usuario no se vea afectada por tareas que no requieren una respuesta inmediata.

La aplicación de pagos realiza una solicitud síncrona al servicio de scoring, que consulta los acumulados en Redis Features, evalúa la transacción y retorna una decisión de aprobación o rechazo en milisegundos. Este flujo es crítico y debe operar con la máxima eficiencia y disponibilidad.

Una vez emitida la decisión, la aplicación publica los datos completos del intento en Redis Streams. De forma desacoplada, el procesador de eventos consume estos mensajes para:

- Actualizar los acumulados en Redis Features.
- Persistir los logs inmutables en Cassandra.
- Disparar alertas automáticas y encolar casos para revisión manual cuando corresponda.

Esta separación asegura que las tareas de alto costo computacional (escritura en disco, actualización de acumulados, generación de alertas) no introduzcan latencia en la respuesta al usuario, y permite que el sistema escale horizontalmente procesando eventos en paralelo.

4. Ciclo de reentrenamiento y feedback (MLOps)

El sistema incorpora un ciclo continuo de mejora del modelo de detección de fraude, basado en el feedback de las decisiones humanas y en la integración de nuevos datos de contracargos. Esta decisión responde a la necesidad de adaptarse a la evolución de los patrones de fraude, que cambian constantemente.

Las decisiones tomadas por los analistas humanos en el flujo de revisión manual, junto con los contracargos confirmados (bloque 5), se consolidan periódicamente en un dataset etiquetado. Este dataset incluye:

- Las transacciones originales (features de entrada).
- La etiqueta de fraude (is_fraud), derivada de la resolución del caso o del contracargo.
- El resultado de las reglas y el scoring emitido en el momento de la evaluación (para retroalimentación del propio motor).

Un proceso batch ETL extrae los datos desde Cassandra y Postgres, los transforma y alimenta un pipeline de entrenamiento automatizado. Este pipeline:

- Entrena nuevos modelos con los datos más recientes.
- Evalúa el desempeño (ej. precisión, recall, AUC-ROC) frente a modelos anteriores.
- Selecciona el modelo con mejor rendimiento para su promoción al registro de modelos.

El modelo seleccionado se registra en el registro de modelos con un identificador de versión único. Desde allí, puede ser promovido a producción y reemplazar al modelo actual en el servicio de scoring, cerrando así el ciclo de mejora continua. Todo el proceso es trazable, permitiendo saber en todo momento qué versión de modelo evaluó cada transacción.

Se monitorean métricas como la tasa de acierto, falsos positivos y falsos negativos del modelo en producción. Estas métricas se retroalimentan al pipeline de entrenamiento para ajustar hiperparámetros y priorizar áreas de mejora en el próximo ciclo.

2. Relevamiento de datos necesarios

Según la descripción de los bloques de datos requeridos, realizada en la sección anterior, a continuación se desglosan los datos que se requiere relevar.

Bloque 1: datos de contexto

Identificadores de entidad (PAGADOR):

- pagador_id: Identificador único del usuario o cliente en el sistema (Clave Primaria).
- tipo_identificacion y identificacion_hash: Tipo y valor cifrado/hasheado del documento de identidad del pagador.
- pais: País de residencia o registro del pagador.
- estado y fecha_alta: Estado operativo del pagador y marca de tiempo de su registro inicial.

Datos de sesión y dispositivo (DISPOSITIVO y PAGADOR_DISPOSITIVO):

- dispositivo_id: Identificador único del dispositivo (Clave Primaria).
- fingerprint: Huella digital del dispositivo generada para la detección de patrones de reuso.
- tipo_dispositivo y sistema_operativo: Tipo de terminal (ej. móvil, escritorio) y sistema operativo detectado.
- primera_observacion y ultima_observacion: Fechas que registran la ventana temporal de relación entre un pagador y un dispositivo específico.

Ubicación y comportamiento acumulado:

- pais_origen: País detectado en el intento de pago durante la transacción (INTENTO_PAGO).

- feature_id, nombre, version, tipo_dato y ventana_temporal: Definición de variables de riesgo e historial acumulado (FEATURE_RIESGO) que se calculan y consultan en tiempo real.

Bloque 2: datos transaccionales

Identificadores y contexto (INTENTO_PAGO):

- intento_id: Identificador único de la transacción o intento de pago (Clave Primaria).
- evento_id: Identificador del evento de origen que dispara la evaluación síncrona.
- fecha_hora: Marca de tiempo exacta del intento de pago.

Datos económicos y del comercio (INTENTO_PAGO y COBRADOR):

- monto y moneda: Monto total de la operación y código de moneda evaluado.
- cobrador_id: Identificador del comercio que procesa la cobranza (Clave Primaria de COBRADOR).
- nom_comercio, categoria y pais: Nombre comercial, rubro o categoría del comercio y país de radicación.

Datos del medio de pago (TARJETA):

- tarjeta_id: Identificador único de la tarjeta utilizada (Clave Primaria).
- pan_token: Token del número de tarjeta para evitar el manejo de datos sensibles (PCI-DSS).
- marca, pais_emisor y estado: Franquicia o marca de la tarjeta (ej. Visa, Mastercard), país del banco emisor y estado del plástico.

Bloque 3: datos de evaluación

Resultados del scoring (DECISION_SCORING):

- decision_id: Identificador único de la evaluación generada (Clave Primaria).
- score_riesgo: Puntuación numérica o score probabilístico emitido por el modelo.
- decision: Veredicto final emitido por el sistema (ej. Aprobado, Rechazado).
- motivo_principal: Código o descripción de la razón principal tras el resultado.
- monto y latencia_ms: Monto evaluado y tiempo en milisegundos que tomó procesar la decisión.

Traza de evaluación (DECISION REGLA, DECISION_FEATURE y MODELO_SCORING):

- modelo_version_id: Identificador exacto de la versión del modelo de ML utilizado para la evaluación.
- regla_version_id, activada, valor_observado y resultado: Traza de ejecución de reglas de negocio (DECISION REGLA) vinculadas a REGLA_FRAUDE.
- feature_id, valor y fecha_calculo_feature: Valores de entrada específicos de las variables de riesgo (DECISION_FEATURE) que alimentaron la decisión.

Bloque 4: datos de gestión y control

Gestión de reglas y configuración (REGLA_FRAUDE):

- regla_version_id: Identificador único de la versión de regla de negocio (Clave Primaria).
- codigo_regla, nombre y expresion_condicion: Código formal, nombre descriptivo y la lógica matemática/condicional de la regla.
- accion, estado y vigencia_desde / vigencia_hasta: Acción a aplicar (ej. rechazar, alertar), estado operativo y período de validez.
- analista_creador_id: Identificador del usuario que creó o modificó la regla.

Auditoría y control por rol (EVENTO_AUDITORIA y ANALISTA_RIESGO):

- evento_auditoria_id: Identificador único del evento de auditoría registrado (Clave Primaria).
- analista_id: Identificador del usuario o analista que ejecutó la acción.
- fecha_hora y accion: Marca de tiempo de la operación y tipo de acción realizada.
- tipo_objeto y objeto_id: Tipo de entidad modificada (ej. Regla, Modelo) y su identificador.
- estado_anterior, estado_nuevo y motivo: Traza del valor anterior, el valor nuevo post-modificación y la justificación dada por el usuario.

Bloque 5: datos de contracargo**Identificadores y datos económicos (CONTRACARGO):**

- contracargo_id: Identificador único del contracargo (Clave Primaria).
- intento_id: Identificador de la transacción original a la que se asocia el contracargo (Clave Foránea hacia INTENTO_PAGO).
- monto: Monto económico disputado en el contracargo.

Causalidad, fechas y estado (CONTRACARGO):

- fecha_reporte: Fecha de notificación del contracargo por parte del procesador o adquirente.
- motivo: Código o descripción del motivo de la disputa provisto por la red de tarjetas.
- estado y fuente: Estado actual en el flujo de gestión del contracargo y la entidad emisora del reporte de contracargo.

Etiquetado para entrenamiento (DATASET_ITEM y DATASET_VERSION):

- etiqueta_fraude: Variable objetivo binaria (\$Y\$) asignada al intento de pago (1 para fraude, 0 para transacción legítima).
- fuente_etiqueta y fecha_maduration: Origen de la confirmación del fraude y fecha de maduración de la etiqueta.
- dataset_version_id: Identificador de la versión del dataset en la que se integró la transacción para el entrenamiento del modelo de ML.

3. Clasificación de los datos según su tipo

A continuación, se presenta la clasificación consolidada de los atributos y variables del modelo de datos, agrupados según su tipo técnico, formato de almacenamiento y su función dentro del sistema:

Tipo de dato técnico	Formato / Dominio	Variables / Atributos del Sistema	Entidades de Origen (Diagramas)	Función en el Sistema
Identificadores y claves (PK / FK)	Identifier / UUID	pagador_id, dispositivo_id, tarjeta_id, cobrador_id, intento_id, decision_id, regla_version_id, modelo_version_id, dataset_version_id, contracargo_id, evento_auditoria_id, analista_id, rol_id, reporte_id, evento_id, analista_creador_id, feature_id, metrica_id	En todas las tablas de Operaciones, Entrenamiento y Reportes	Garantizar la integridad referencial, la trazabilidad entre entidades y la resolución de identidad.
Tokens y hashes de identidad	String (Hash / Token)	identificacion_hash, pan_token, fingerprint	PAGADOR, TARJETA, DISPOSITIVO	Anonimización bajo cumplimiento normativo (PCI-DSS / GDPR) y detección de reuso de dispositivos.
Numéricos continuos	Decimal / Float	monto, score_riesgo, valor	INTENTO_PAGO, DECISION_SCORING, DECISION_FEATURE, METRICA_MODELO, CONTRACARGO	Representación de valores económicos, probabilidades continuas de fraude y métricas de desempeño del modelo.
Numéricos discretos	Integer / BigInt	latencia_ms, valor_observado	DECISION_SCORING, DECISION_REGLA	Medición del rendimiento técnico (ms) y evaluación de acumulados frente a umbrales de reglas.
Categoricos nominales (entidades y geografía)	String (Código / Nom)	tipo_identificacion, pais, pais_origen, pais_emisor, marca, categoria, tipo_dispositivo, sistema_operativo	PAGADOR, TARJETA, COBRADOR, DISPOSITIVO, INTENTO_PAGO	Caracterización del contexto transaccional y variables cualitativas para <i>feature engineering</i> .

Categoricos nominales (estados y decisiones)	String (Enum)	estado, decision	PAGADOR, TARJETA, COBRADOR, REGLA_FRAUDE, CONTRACARGO, MODELO_SCORING, DATASET_VERSION, DECISION_SCORING	Control del ciclo de vida operativo de entidades y registro del veredicto del scoring (ej. Aprobado, Rechazado).
Categoricos nominales (causalidad y fuentes)	String (Código / Categoría)	motivo_principal, motivo, fuente, fuente_etiqueta, tipo_objeto, tipo_dato, ventana_temporal	DECISION_SCORING, CONTRACARGO, DATASET_ITEM, EVENTO_AUDITORIA, FEATURE_RIESGO	Clasificación de causas de rechazo/contracargo, trazabilidad de auditoría y parametrización de variables.
Temporales (marcas de evento)	Datetime (ISO 8601 UTC)	fecha_hora, fecha_calculo, fecha_calculo_feature	INTENTO_PAGO, EVENTO_AUDITORIA, METRICA_MODELO, DECISION_FEATURE	Registro del momento exacto de ejecución transaccional, auditorías y evaluaciones de modelos.
Temporales (ciclo de vida y rangos)	Datetime (ISO 8601 UTC)	fecha_alta, primera_observacion, ultima_observacion, fecha_creacion, fecha_generacion, vigencia_desde, vigencia_hasta, ventana_desde, ventana_hasta, fecha_corte_labels, fecha_maduracion, fecha_reporte	PAGADOR, ANALISTA_RIESGO, DISPOSITIVO, PAGADOR_DISPOSITIVO, REGLA_FRAUDE, DATASET_VERSION, DATASET_ITEM, CONTRACARGO	Definición de ventanas temporales para el recálculo histórico, maduración de etiquetas (\$Y\$) e historiales de uso.
Lógicos / booleanos	Boolean (1 / 0)	activada, etiqueta_fraude, activo	DECISION_REGLA, DATASET_ITEM, FEATURE_RIESGO	Banderas binarias de activación de reglas, disponibilidad de variables y variable objetivo (\$Y\$) para entrenamiento de ML.
Texto estructurado / lógica	Text (Expresión / Código)	expresion_condicion, codigo_regla	REGLA_FRAUDE	Definición de sintaxis ejecutable de reglas de negocio y códigos de regla.

Texto descriptivo y auditoría	Text / String	nom_comercio, nombre, descripcion, email, accion, estado_anterior, estado_nuevo, motivo	COBRADOR, ROL, FEATURE_RIESGO, REGLA_FRAUDE, MODELO_SCORING, REPORTE, ANALISTA_RIESGO, EVENTO_AUDITORIA	Trazabilidad del control de acceso (RBAC), nombres informativos y traza del estado previo y posterior en auditorías.
--------------------------------------	---------------	---	---	--

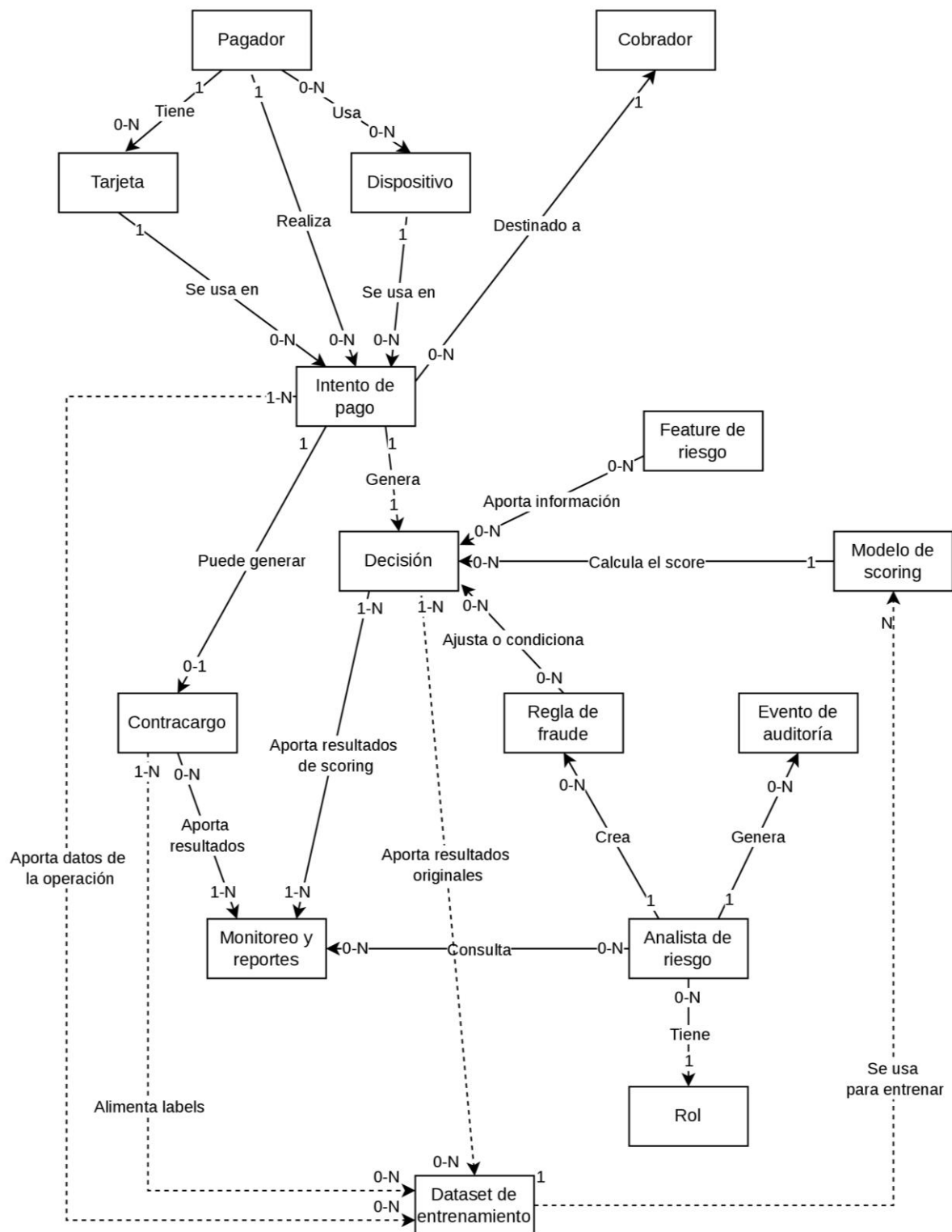
3.1. Matriz de Mapeo de los datos

A continuación se presenta un esquema de cómo se distribuyen los bloques de datos entre los distintos motores de bases de datos según la arquitectura planteada:

Bloque de Datos	Motor de Almacenamiento	Estructura / Modelo Técnico	Razón de Elección Tecnológica
Bloque 1: Contexto (Redis)	Redis Features (<i>In-Memory</i>)	HASH de Redis (user:{id}:features)	Latencia extremadamente baja para lectura síncrona en el servicio de scoring.
Bloque 1 y 2: Streaming Eventos	Redis Streams (<i>Bus de eventos</i>)	STREAM de Redis (stream:events:payments)	Ingesta asíncrona de alto rendimiento para desacoplar el scoring del procesador de eventos.
Bloque 1, 2, 3: Logs inmutables	Cassandra (<i>NoSQL Columnar</i>)	Tabla distribuida ordenada por (pagador_id, fecha_hora)	Alta velocidad de escritura, escalabilidad horizontal y persistencia histórica inmutable.
Bloque 1 a 5: Datos Relacionales	PostgreSQL (<i>SQL Relacional</i>)	Tablas relacionales con \$PK\$/FK\$ (Modelos de Operaciones, Entrenamiento y Reportes)	Soporte ACID, consultas complejas con JOINS, agregaciones batch, etiquetado de entrenamiento y reporting.

4. Modelo conceptual

A continuación, se consolida el modelo conceptual del dominio, para representar las reglas de negocio, los actores, las entidades clave y las interacciones del sistema sin dependencia de la implementación tecnológica final. Este marco garantiza la integridad de la información desde la captura del intento de pago hasta la maduración de los modelos analíticos y los controles de auditoría.



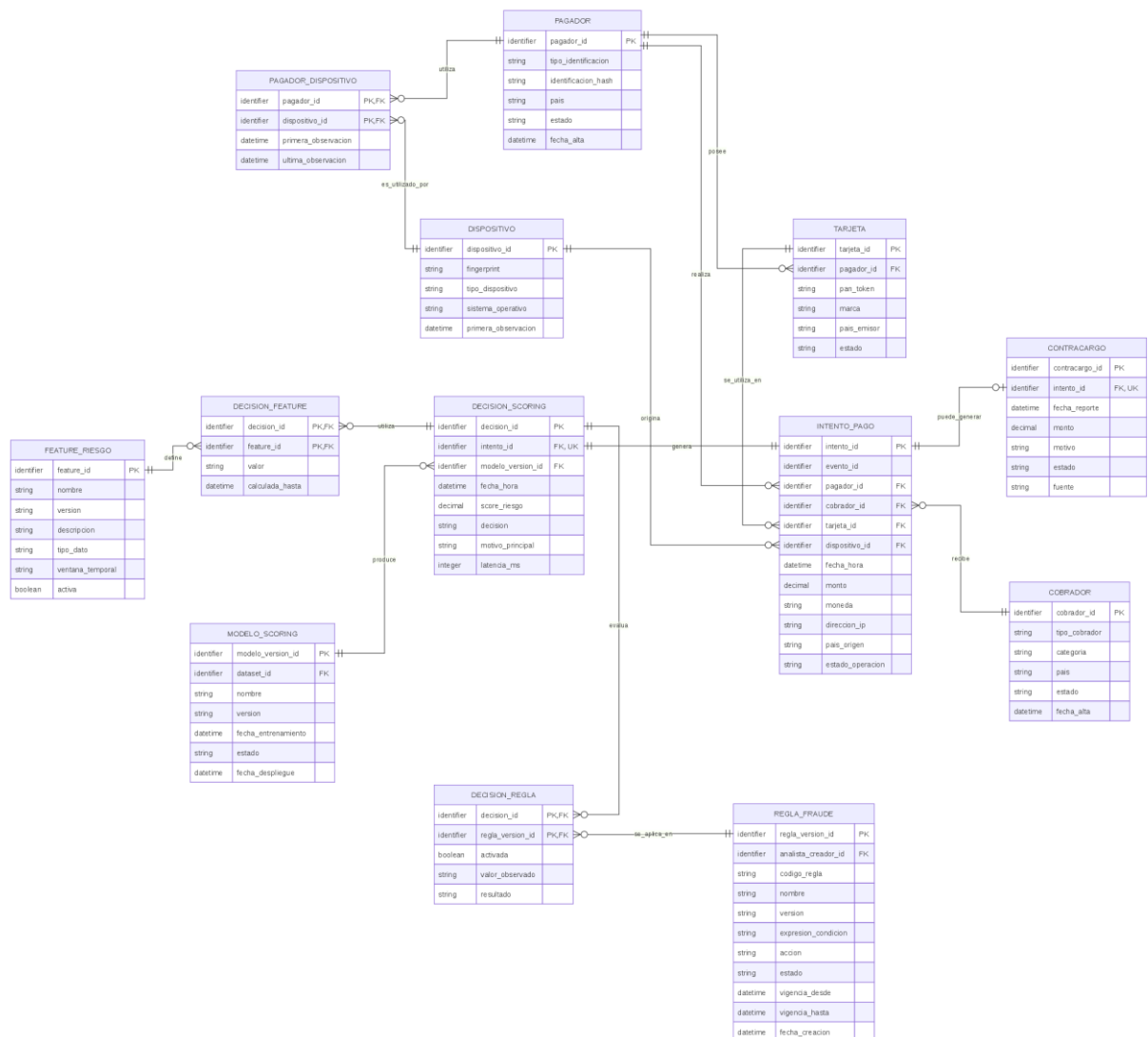
5. Modelo de implementación según la tecnología elegida

Para representar la implementación del modelo conceptual en la capa relacional, se diseñan tres esquemas lógicos según las necesidades de persistencia permanente en PostgreSQL, los cuales son: 1. Modelo de operaciones, 2. Modelo de entrenamiento (MLOps) y 3. Modelo de reportes y auditoría. Cabe aclarar que estos tres componentes constituyen la arquitectura de persistencia transaccional y analítica estructurada. Por su parte, el diseño de las estructuras para Redis, Redis Streams y Cassandra se detalla en el punto 11 (Propuesta para datos semiestructurados, no estructurados o vectoriales).

5.1. Modelo de operaciones

Para ver el diagrama con mejor resolución puede ir al siguiente enlace:

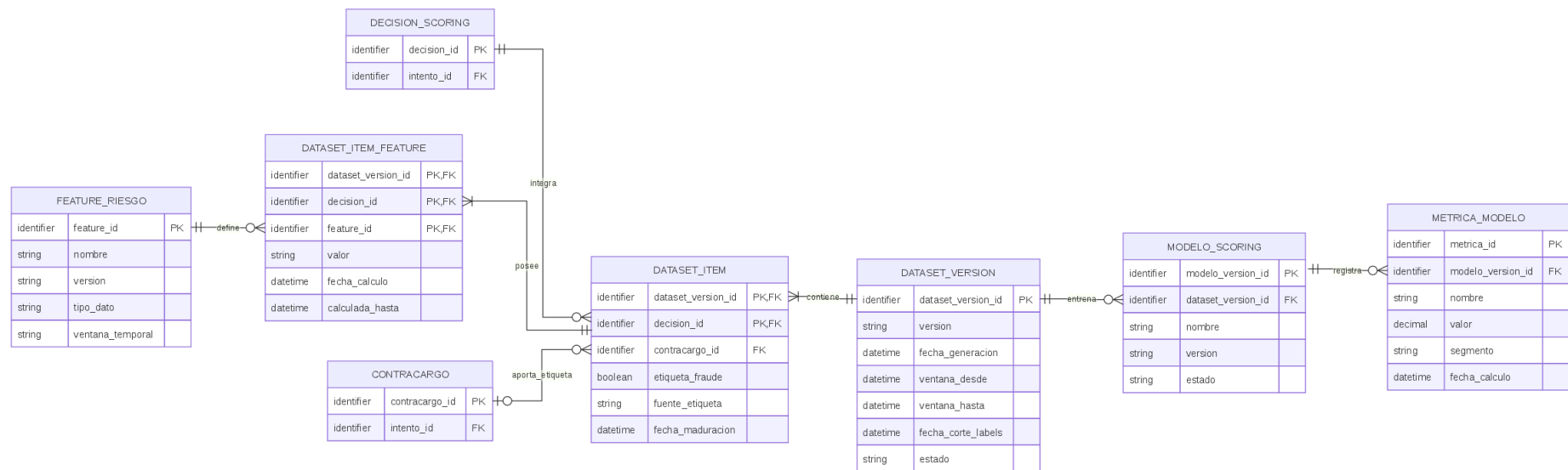
[https://github.com/jpskobalski/tp_bases_de_datos_ai/blob/main/tp_integrador/docs/Diagrama logico operaciones.pdf](https://github.com/jpskobalski/tp_bases_de_datos_ai/blob/main/tp_integrador/docs/Diagrama%20logico_operaciones.pdf)



5.2. Modelo de entrenamiento (MLOps)

Para ver el diagrama con mejor resolución puede ir al siguiente enlace:

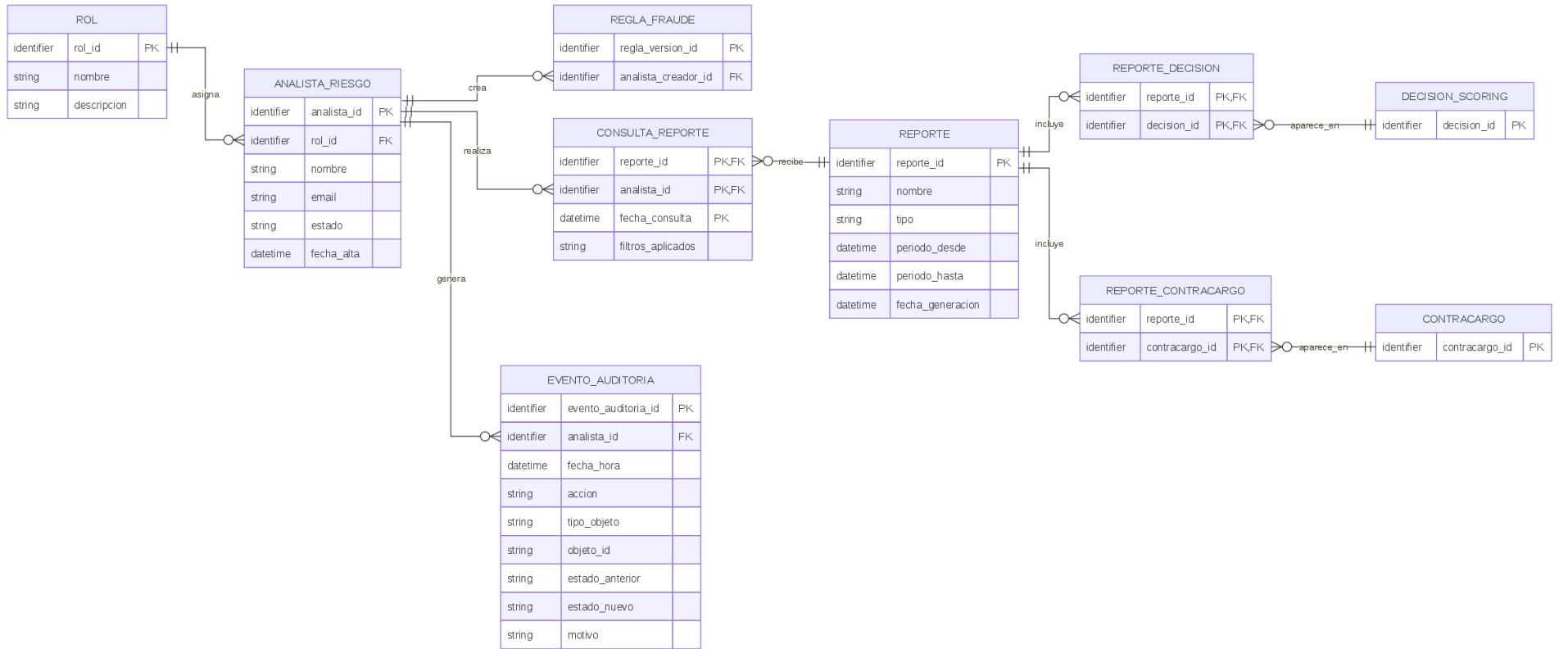
https://github.com/jpskobalski/tp_bases_de_datos_ai/blob/main/tp_integrador/docs/Diagrama_logico_entrenamiento.pdf



5.3. Modelo de reportes

Para ver el diagrama con mejor resolución puede ir al siguiente enlace:

https://github.com/jpskobalski/tp_bases_de_datos_ai/blob/main/tp_integrador/docs/Diagrama_logico_reportes..pdf



6. Decisiones de normalización, embebido, referencia o desnormalización

La arquitectura del sistema responde a un diseño híbrido fundamentado en el principio de persistencia políglota. Esta estrategia asigna cada grupo de datos al motor de base de datos cuyo paradigma técnico resuelve de forma eficiente las restricciones operativas de la aplicación.

6.1. Modelo relacional (PostgreSQL)

El modelo relacional implementado en PostgreSQL adopta la tercera forma normal (3FN) para la totalidad de las tablas en las capas de Operaciones, MLOps y Reportes. Esta decisión prioriza la integridad referencial, el rigor de la estructura y la consistencia transaccional.

Los principales problemas que se pretenden evitar son:

- Redundancia de datos: la separación explícita de las entidades maestras como PAGADOR, TARJETA, COBRADOR, DISPOSITIVO, frente a las tablas de eventos, por ejemplo INTENTO_PAGO, DECISION_SCORING, evita la duplicación de atributos descriptivos en cada registro de evaluación.
- Anomalías de inserción: el diseño permite el alta de un nuevo pagador o dispositivo sin la obligación de registrar de forma simultánea un intento de pago o una transacción.
- Anomalías de actualización: la modificación del estado de una tarjeta o la actualización del rubro de un comercio se ejecuta en un único registro maestro. La actualización se refleja en todas las consultas futuras sin generar inconsistencias.
- Anomalías de eliminación: la baja de una regla de fraude o la desactivación de un usuario analista no provoca la pérdida del historial de evaluaciones ni de los registros de auditoría asociados.

Las estructuras para MLOps y auditoría requieren aislamiento estricto de transacciones. El cumplimiento de las propiedades ACID en PostgreSQL impide que la creación paralela de conjuntos de entrenamiento o la modificación de roles altere la trazabilidad del sistema.

6.2. Modelos NoSQL y en memoria

Para responder a las demandas de latencia en milisegundos y al procesamiento de millones de eventos diarios, el sistema incorpora dos componentes NoSQL desnormalizados.

Por un lado, Redis Features, la cual es una arquitectura tipo clave-valor en memoria, se usa la desnormalización total basada en el patrón de agregación previa. Esto, debido a que el servicio de scoring requiere acceso a métricas históricas acumuladas en un tiempo inferior a 5 milisegundos. La ejecución de operaciones de agregación (por ejemplo: sumas de montos o conteos de frecuencia) sobre una base relacional en cada intento de pago generaría cuellos de botella insostenibles. Esta decisión acepta la duplicación de contadores entre diferentes estructuras Hash (PAYER, CARD, DEVICE, COLLECTOR) a cambio de lecturas directas con complejidad $O(1)$.

Por otro lado, Cassandra, el cual es una arquitectura tipo columnar distribuido y de escritura continua, se implementó un diseño orientado a la consulta (Query-First Design) con alta desnormalización. Esto se debe a que la tabla PAYMENT_DECISION_LOG_BY_DAY empaqueta en una única fila lógica los datos del intento, el snapshot de variables (feature_snapshot) y las reglas evaluadas (reglas_disparadas). Esta arquitectura de escritura continua elimina la contención por bloqueos de fila y permite la ingesta masiva de registros con disponibilidad alta. A pesar de sus ventajas, para el caso de uso, se acepta la imposibilidad de

realizar JOINS y la consistencia eventual entre los nodos del clúster. La inmutabilidad de los registros mitiga los riesgos de inconsistencia por modificaciones posteriores. Sin embargo, esto se mitiga con el respaldo en PostgreSQL.

6.3. Cobertura de los patrones de consulta del dominio

La combinación de esquemas estructurados y desnormalizados satisface los requerimientos clave del caso de uso:

Patrón de consulta	Tecnología seleccionada	Mecanismo y estructura	Beneficio alcanzado
Evaluación síncrona de riesgo	Redis Features	Consulta por clave HASH	Latencia ultrabaja ($< 5\text{ ms}$)
Ingesta e inmutabilidad de logs	Cassandra	Particionamiento por fecha y <i>bucket</i>	Escritura masiva sin bloqueos
Revisión de casos y auditoría	PostgreSQL (Operaciones Reportes)	Consultas relacionales con JOINS	Consistencia ACID y trazabilidad
Construcción de datasets de ML	PostgreSQL (MLOps)	Cruce batch con tablas de contracargo	Etiquetado preciso y reproducible

7. Justificación de la tecnología seleccionada

La naturaleza del caso de uso exige resolver dos restricciones contrapuestas: la evaluación síncrona con latencias en milisegundos para la toma de decisiones y la persistencia inmutable de millones de transacciones diarias para procesos analíticos y de auditoría. Un único motor de base de datos resulta insuficiente para satisfacer ambos extremos. Por este motivo, se selecciona un enfoque de persistencia políglota que combina tres tecnologías: PostgreSQL, Redis y Cassandra.

7.1. PostgreSQL

PostgreSQL asume el rol de motor transaccional primario para la gestión de datos altamente estructurados que demandan un esquema rígido y validaciones estrictas de integridad. Esta tecnología garantiza un nivel de consistencia estricta bajo el cumplimiento de las propiedades ACID, aspecto indispensable para la administración de las operaciones de negocio, el control de acceso basado en roles, la configuración de reglas de fraude y el registro de eventos de auditoría. Su modelo relacional permite resolver los vínculos complejos de uno a muchos (1:N) y muchos a muchos (N:M) entre pagadores, tarjetas, comercios, usuarios, reglas y versiones de modelos de inteligencia artificial. Asimismo, administra la seguridad mediante la restricción de permisos y el uso de vistas para aislar la información sensible bajo normativas como PCI-DSS y GDPR.

Entre sus principales ventajas destaca la capacidad para evitar anomalías operativas y asegurar la consistencia en el etiquetado de los datasets de entrenamiento. Sin embargo, asume como

limitación un costo elevado de escalabilidad vertical y una degradación en el rendimiento frente a consultas analíticas complejas sobre volúmenes masivos en tiempo real.

7.2. Redis Features y Streams

Redis opera como la capa en memoria orientada a responder a las demandas de latencia crítica en el flujo online. A través de estructuras tipo Hash, Redis Features almacena los acumulados históricos de riesgo por entidad, mientras que Redis Streams gestiona secuencias temporales para la ingesta asíncrona de eventos. Este diseño permite lecturas directas por clave con complejidad $O(1)$ y tiempos de respuesta inferiores a 5 milisegundos, lo cual mantiene la estabilidad del servicio de scoring ante picos de demanda al eliminar por completo los accesos a disco. Además, el sistema escala horizontalmente mediante técnicas de fragmentación y clústeres en memoria.

La adopción de esta tecnología ofrece la ventaja de desacoplar el motor de evaluación de los procesos posteriores de persistencia. Como compromiso técnico, esta decisión exige una inversión considerable en memoria RAM y acepta la volatilidad del almacenamiento, razón por la cual la conservación de los registros históricos se delega a Cassandra.

7.3. Cassandra

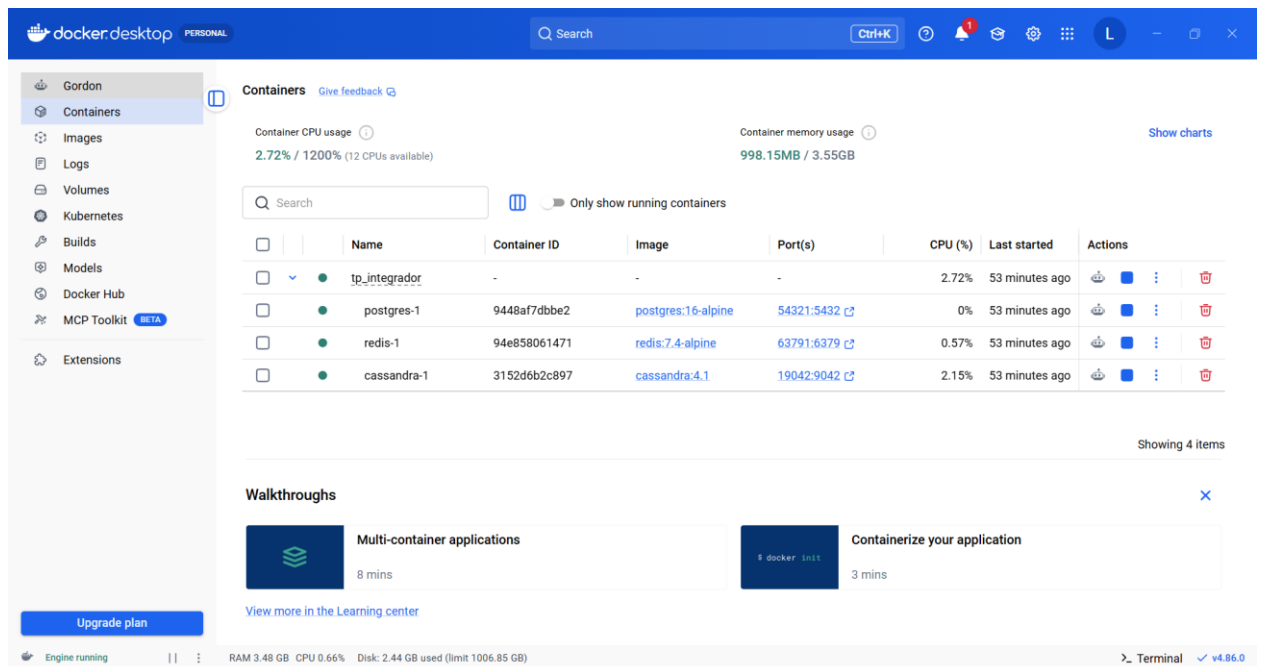
Cassandra constituye la solución de persistencia distribuida orientada a columnas (Wide-Column), seleccionada de forma específica para soportar la ingesta masiva e ininterrumpida de información. Su arquitectura ofrece una escalabilidad horizontal lineal mediante la incorporación de nodos al clúster sin puntos únicos de falla, lo que otorga la capacidad de procesar millones de eventos diarios.

El motor opera bajo un esquema de consistencia eventual, decisión de diseño que prioriza la velocidad de escritura y la alta disponibilidad del clúster. Dentro del esquema del sistema, la tabla `PAYMENT_DECISION_LOG_BY_DAY` organiza los registros mediante claves de partición (`fecha_particion`, `bucket`) y claves de ordenamiento (`fecha_hora`, `event_id`), estructura que optimiza tanto la ingesta masiva continua como las extracciones periódicas en lote (`batch`).

La principal ventaja de esta elección radica en su tolerancia a altas tasas de escritura sin sufrir contención por bloqueos de fila o tabla. En contraposición, el modelo sacrifica la capacidad de ejecutar JOINS y consultas ad-hoc que no se encuentren previstas dentro del diseño original de las claves de partición.

8. Implementación mínima realizada

La validación práctica de la solución se ejecuta mediante un entorno local en containers de Docker, que despliegan los motores de PostgreSQL, Redis y Cassandra, realiza la carga de datos sintéticos, ejecuta la suite de consultas representativas y procesa transacciones en tiempo real. A continuación, se muestra la implementación de los containers en Docker. En el repositorio del trabajo se encuentra todo el proyecto listo para ser ejecutado: .



8.1. Arquitectura implementada

La infraestructura local se orqueste mediante Docker Desktop y un archivo Make que automatiza las etapas de provisión, verificación e inferencia.

1. PostgreSQL: entidades operacionales, reglas, decisiones, contracargos, auditoría, reportes y datasets de entrenamiento.
2. Redis: hashes de features online, relaciones de cardinalidad y Redis Streams como bus de eventos.
3. Cassandra: logs append-only y tablas desnormalizadas orientadas a consultas por fecha, pagador, comercio y decisión.
4. Python: servicio de scoring, Random Forest, publicación del evento y procesador asíncrono.

La aplicación real implementa el camino Scoring Service -> Redis Features -> Redis Streams -> Event Processor -> Cassandra. PostgreSQL representa la capa relacional alimentada por el proceso batch de la arquitectura.

El despliegue de los servicios se gestiona a través de Docker Compose, el cual levanta de forma aislada los contenedores de PostgreSQL, Redis y Cassandra. Cada servicio cuenta con verificaciones de estado de salud (health checks) para verificar la disponibilidad de los puertos y motores antes de iniciar las operaciones de configuración.

```
PS C:\Users\yuran\OneDrive\Documents\Luis\1_MIAE\5_BD_IA\tp_bases_de_datos_ai\tp_integrador> make todo
docker compose up -d --wait --wait-timeout 240
[+] up 42/42
✓ Image postgres:16-alpine Pulled 57.2s
✓ Image redis:7.4-alpine Pulled 45.3s
✓ Image cassandra:4.1 Pulled 41.8s
✓ Network tp_integrador_default Created 0.1s
✓ Volume tp_integrador_postgres_data Created 0.0s
✓ Volume tp_integrador_redis_data Created 0.0s
✓ Volume tp_integrador_cassandra_data Created 0.0s
✓ Container tp_integrador-redis-1 Healthy 7.0s
✓ Container tp_integrador-cassandra-1 Healthy 73.0s
✓ Container tp_integrador-postgres-1 Healthy 7.1s
./scripts/inicializar_nosql.sh
process_begin: CreateProcess(NULL, env bash C:\Users\yuran\OneDrive\Documents\Luis\1_MIAE\5_BD_IA\tp_bases_de_datos_ai\tp_integrador\scripts\inicializar_nosql.sh, ...) failed.
make (e=2): El sistema no puede encontrar el archivo especificado.
make: *** [Makefile:11: inicializar] Error 2
```

Los puertos mapeados en el entorno local corresponden a 54321 para PostgreSQL, 63791 para Redis y 19042 para Cassandra.

A continuación, se muestran algunas capturas de pantalla donde se muestran los logs de la inicialización y uso de cada motor:

Containers / tp_integrador-redis-1

tp_integrador-redis-1

94e858061471

redis:7.4-alpine

63791:6379

STATUS

Running (1 hour ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

1:C 16 Aug 2026 12:41:27.209 * o000o000o000o Redis is starting o000o000o000o

1:C 16 Aug 2026 12:41:27.209 * Redis version=7.4.10, bits=64, commit=00000000, modified=0, pid=1, just started

1:C 16 Aug 2026 12:41:27.209 * Configurattion loaded

1:M 16 Aug 2026 12:41:27.210 * monotonic clock: POSIX clock_gettime

1:M 16 Aug 2026 12:41:27.213 * Running mode=standalone, port=6379.

1:M 16 Aug 2026 12:41:27.214 * Server initialized

1:M 16 Aug 2026 12:41:27.229 * Creating AOF base file appendonly.aof.1.base.rdb on server start

1:M 16 Aug 2026 12:41:27.250 * Creating AOF incr file appendonly.aof.1.incr.aof on server start

1:M 16 Aug 2026 12:41:27.250 * Ready to accept connections tcp

Containers / tp_integrador-cassandra-1

tp_integrador-cassandra-1

3152d6b2c897

cassandra:4.1

19042:9042

STATUS

Running (1 hour ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

INFO [ScheduledTasks:1] 2026-08-16 13:41:35,042 ColumnFamilyStore.java:1015 - Enqueueing flush of system.sstable_activity_v2, Reason: MEMTABLE_PERIOD_EXPIRED, Usage: 2.194KiB (0%) on-heap, 0.000KiB (0%) off-heap

INFO [PerDiskMemtableFlushWriter_0:4] 2026-08-16 13:41:35,060 Flushing.java:145 - Writing Memtable-sstable_activity_v2@2108854647(0.055KiB serialized bytes, 7 ops, 2.194KiB (0%) on-heap, 0.000KiB (0%) off-heap), flushed range = [min(-9223372036854775808), max(9223372036854775807))

INFO [PerDiskMemtableFlushWriter_0:4] 2026-08-16 13:41:35,062 Flushing.java:171 - Completed Flushing /var/lib/cassandra/data/system/sstable_activity_v2-62efe31f3be8310c8d298963439c1288/nb-1-big-Data.db (0.246KiB) for commitlog position CommitLogPosition(segmentId=1786884092286, position=631526)

INFO [ScheduledTasks:1] 2026-08-16 13:41:35,356 ColumnFamilyStore.java:1015 - Enqueueing flush of system.size_estimates, Reason: MEMTABLE_PERIOD_EXPIRED, Usage: 132.227KiB (0%) on-heap, 0.000KiB (0%) off-heap

INFO [PerDiskMemtableFlushWriter_0:3] 2026-08-16 13:41:35,383 Flushing.java:145 - Writing Memtable-size_estimates@460190057(22.470KiB serialized bytes, 2178 ops, 132.227KiB (0%) on-heap, 0.000KiB (0%) off-heap), flushed range = [min(-9223372036854775808), max(9223372036854775807))

INFO [ScheduledTasks:1] 2026-08-16 13:41:35,424 ColumnFamilyStore.java:1015 - Enqueueing flush of system.table_estimates, Reason: MEMTABLE_PERIOD_EXPIRED, Usage: 287.467KiB (0%) on-heap, 0.000KiB (0%) off-heap

INFO [PerDiskMemtableFlushWriter_0:4] 2026-08-16 13:41:35,449 Flushing.java:145 - Writing Memtable-table_estimates@2039286297(48.568KiB serialized bytes, 4356 ops, 287.467KiB (0%) on-heap, 0.000KiB (0%) off-heap), flushed range = [min(-9223372036854775808), max(9223372036854775807))

INFO [PerDiskMemtableFlushWriter_0:3] 2026-08-16 13:41:35,461 Flushing.java:171 - Completed flushing /var/lib/cassandra/data/system/size_estimates-618f817b005f3678b8a453f3930b8e86/nb-1-big-Data.db (16.056KiB) for commitlog position CommitLogPosition(segmentId=1786884092286, position=631526)

INFO [PerDiskMemtableFlushWriter_0:4] 2026-08-16 13:41:35,474 Flushing.java:171 - Completed Flushing /var/lib/cassandra/data/system/table_estimates-176c39cdb93d33a5a2188eb06a56f66e/nb-1-big-Data.db (36.517KiB) for commitlog position CommitLogPosition(segmentId=1786884092286, position=631526)

INFO [IndexSummaryManager:1] 2026-08-16 13:41:36,037 IndexSummaryRedistribution.java:87 - Redistributing index summaries

INFO [IndexSummaryManager:1] 2026-08-16 13:41:36,040 IndexSummaryRedistribution.java:98 - Beginning redistribution of index summaries for 22 sstables with memory pool size 25 MiB; current spaced used is 5.426406860351562E-4 MiB

INFO [IndexSummaryManager:1] 2026-08-16 13:41:36,052 IndexSummaryRedistribution.java:250 - index summaries: downsample: 0, force resample: 0, upsample: 0, force upsample: 0

INFO [IndexSummaryManager:1] 2026-08-16 13:41:36,072 IndexSummaryRedistribution.java:137 - Completed resizing of index summaries; current approximate memory used: 5.426406860351562E-4 MiB, time spent: 35ms

Containers / tp_integrador-postgres-1

tp_integrador-postgres-1

9448af7dbbe2 postgres:16-alpine 54321:5432

STATUS
Running (1 hour ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

The files belonging to this database system will be owned by user "postgres". This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8". The default database encoding has accordingly been set to "UTF8". The default text search configuration will be set to "english".

Data page checksums are disabled.

```
fixing permissions on existing directory /var/lib/postgresql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... UTC
creating configuration files ... ok
running bootstrap script ... ok
sh: locale: not found
2026-08-16 12:41:28.050 UTC [35] WARNING: no usable system locales were found
performing post-bootstrap initialization ... ok
initdb: warning: enabling "trust" authentication for local connections
```

8.2. Flujo de orquestación

El flujo de orquestación se inicia con el comando de ejecución global **make todo**, que desencadena las siguientes acciones secuenciales:

1. levanta PostgreSQL, Redis y Cassandra y espera sus health checks;
2. crea esquemas, índices y vistas;
3. carga datos sintéticos deterministas;
4. ejecuta 8 consultas PostgreSQL, 5 Cassandra y 8 operaciones Redis;
5. instala las dependencias Python en .venv;
6. ejecuta tres pagos contra Redis y Cassandra reales.

Para reconstruir también los volúmenes desde cero se usa el comando: **make reiniciar**

Para detener los servicios sin borrar datos se usa el comando: **make detener**

8.3. Creación de tablas, registros y consultas

A continuación, se muestran los resultados de la creación de tablas, registros y consultas realizadas:

Creación de tablas:

tp_integrador-postgres-1

 9448af7dbbe2   postgres:16-alpine

54321:5432 

Logs Inspect Bind mounts Exec Files Stats

done
server started
CREATE DATABASE

```
/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01_operaciones.sql
CREATE EXTENSION
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

tp_integrador-postgres-1

 9448af7dbbe2   postgres:16-alpine

54321:5432 

STATUS
Running

Logs Inspect Bind mounts Exec Files Stats

```
/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/02_reportes.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE

/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/03_entrenamiento.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE

/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/04_relaciones_cruzadas.sql
ALTER TABLE
ALTER TABLE

/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/05_indices_vistas.sql
CREATE INDEX
CREATE INDEX
```

RAM 3.49 GB CPU 0.33% Disk: 2.44 GB used (limit 1006.85 GB)

Inserción de datos sintéticos


tp_integrador-postgres-1

 9448af7dbbe2
  postgres:16-alpine

 54321:5432

[Logs](#)
[Inspect](#)
[Bind mounts](#)
[Exec](#)
[Files](#)
[Stats](#)

CREATE TABLE

CREATE INDEX

```

/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/06_datos_sinteticos.sql
BEGIN
INSERT 0 3
INSERT 0 3
INSERT 0 1
INSERT 0 5
INSERT 0 4
INSERT 0 1
INSERT 0 5
INSERT 0 5
INSERT 0 3
INSERT 0 5
INSERT 0 7
INSERT 0 12
INSERT 0 12
INSERT 0 3
INSERT 0 60
INSERT 0 9
INSERT 0 12
INSERT 0 60
INSERT 0 3
INSERT 0 2
INSERT 0 3

```

8.3. Scripts para la implementación de la solución

El flujo de datos y la arquitectura de la solución se implementó a través de dos scripts o módulos de Python, para evaluar la lógica del dominio tanto de forma aislada en memoria como sobre los motores de persistencia distribuidos. Estos módulos se encuentran en la ruta: `tp_integrador/app`

- `mock_fraud_pipeline.py`
- `real_online_pipeline.py`

El módulo `mock_fraud_pipeline.py` simula un entorno de prueba unitario y autocontenido que encapsula las estructuras de datos, las reglas de negocio y el modelo de riesgo de aprendizaje automático. El módulo ejecuta todo el flujo de decisión dentro del proceso de Python. Esta implementación no usa conexiones de red y permite validar que las reglas y del algoritmo de Random Forest sean correctos sin requerir los servicios contenerizados.

El módulo `real_online_pipeline.py` es el entorno de integración sobre la infraestructura física del trabajo. Este script gestiona los clientes de conexión hacia los puertos locales de Redis y Cassandra, construye los pipelines de comandos atómicos para la actualización de variables, ejecuta consultas preparadas en la base de datos distribuida y gestiona la confirmación de mensajes en el bus de eventos.

El módulo `mock_fraud_pipeline.py` provee la regla de decisión y el cálculo de la probabilidad de fraude; por otro lado, `real_online_pipeline.py` proporciona acceso a la infraestructura física de producción en Docker. Cuando se ejecuta el pipeline real, el sistema lee el contexto desde los hashes de Redis, evalúa la transacción con la lógica compartida de `mock_fraud_pipeline.py` y emite la respuesta síncrona. De forma asíncrona, el procesador

de `real_online_pipeline.py` consume el evento desde el stream de Redis, guarda la información en las tablas físicas de Cassandra y confirma el mensaje en el bus de eventos.

Integración y rol de cada motor durante la transacción

La prueba ejecuta una secuencia de tres transacciones de pago sobre el entorno contenerizado, en cada transacción intervienen los tres motores:

- PostgreSQL se inicializa de forma previa mediante los scripts DDL y de carga sintética durante el despliegue del contenedor. Aporta el marco de referencia relacional para contratos, datos de entidades y la posterior auditoría o entrenamiento de modelos en lote (batch).
- Redis participa en el camino síncrono y asíncrono. En el camino síncrono, provee el perfil de variables en tiempo real, luego de emitir el veredicto registra el evento en el stream `fraud:payment_scored` mediante la orden `XADD`. En el camino asíncrono, el consumidor procesa el stream con `XREADGROUP` y actualiza atómicamente los hashes y conjuntos de relaciones (Sets) con `HINCRBY` y `SADD`.
- Cassandra opera en el camino asíncrono a través del proceso consumidor. Este verifica primero la idempotencia en la tabla `eventos_procesados` y luego guarda de forma simultánea el evento en las tablas de consulta inmutables:
`payment_decision_log_by_day,` `intentos_pago_por_pagador,`
`intentos_pago_por_comercio,` `decisiones_scoring` `y`
`evaluaciones_features.`

9. Datos de ejemplo utilizados

Toda la información del sistema, tales como las transacciones, reglas, etiquetas y variables son puramente sintéticas. El modelo de aprendizaje automático basado en Random Forest se entrena durante la ejecución del proceso mediante vectores generados de forma determinista, con el propósito exclusivo de validar los contratos de datos y la integración entre componentes.

La generación e inyección de datos de prueba se distribuye entre los scripts de Python y los archivos de inicialización SQL:

- El módulo de simulación `mock_fraud_pipeline.py` encapsula el generador de vectores sintéticos en memoria y proporciona los métodos `seed()` para poblar las variables base en estructuras de diccionarios en RAM.
- El módulo de integración `real_fraud_pipeline.py` reutiliza la lógica de generación del primer módulo e invoca el método `seed()` del adaptador `RedisFeatureStore` para inyectar las claves y métricas iniciales directamente sobre las instancias reales de Redis.
- La base de datos relacional `01_datos_sinteticos.sql`, que se encuentra en la ruta `\tp_integrador\db\datos`, carga los datos tabulares iniciales en PostgreSQL, compuestos de 12 intentos de pago, decisiones, contracargos y métricas de modelo, todo esto mediante el archivo SQL de inicialización durante el arranque del contenedor de Docker, independientemente de la ejecución de los scripts de Python.

Para ajustar el algoritmo de Random Forest, el pipeline real importa la función `_synthetic_training_data()` de la clase `FraudRiskModel`, la cual genera un conjunto de 1500 vectores de entrenamiento reproducibles mediante una semilla fija. De forma complementaria, el flujo invoca la función `make_request()` del módulo simulado para construir los tres intentos de pago sintéticos que evalúan pipeline en tiempo real.

10. Consultas representativas

Para el trabajo, se definieron operaciones y consultas automatizadas en los tres motores utilizados. A continuación se describen para cada uno de ellos:

Las operaciones y comandos de Redis para el flujo transaccional se estructuran según el componente y la estructura de datos utilizada:

8.1. Operaciones sobre Hashes y Sets en RedisFeature

- `hset` / `hgetall`: Inicialización de perfiles predeterminados en la función `seed` y lectura completa del perfil de variables en tiempo real.
- `hincrby` / `hincrbyfloat`: Incremento atómico de contadores transaccionales y acumulación monetaria sobre los hashes de las entidades.
- `sadd` / `scard`: Registro de relaciones unívocas en conjuntos y cálculo de cardinalidad para determinar el número de entidades distintas en las ventanas temporales.
- `pipeline`: Agrupación atómica de comandos `hincrby`, `hincrbyfloat`, `hset` y `sadd` para garantizar ejecuciones en un solo viaje de red (round-trip).

8.1.1. Operaciones el bus de eventos en RedisStreamEventBus y RedisStreamEventProcessor

- `xadd`: Publicación del evento transaccional serializado dentro del stream de evaluación.
- `xreadgroup`: Lectura asíncrona por parte del grupo consumidor para el procesamiento fuera del camino síncrono.
- `xack`: Confirmación de procesamiento exitoso y liberación del evento en la lista de pendientes (PEL).

8.1.2. Algunas consultas representativas para Redis:

- Ver las métricas / features calculadas para un pagador
Inspecciona todo el Hash del pagador 7002:

```
$ docker compose exec redis redis-cli
127.0.0.1:6379> KEYS fraud:*
1) "fraud:relations:payer:00000000-0000-0000-0000-000000007002:devices"
2) "fraud:features:collector:00000000-0000-0000-0000-000000009002"
3) "fraud:relations:device:00000000-0000-0000-0000-000000008005:payers"
4) "fraud:features:card:00000000-0000-0000-0000-00000000a002"
5) "fraud:features:device:00000000-0000-0000-0000-000000008005"
6) "fraud:payment_scored"
7) "fraud:features:payer:00000000-0000-0000-0000-000000007002"
8) "fraud:features:payer:00000000-0000-0000-0000-000000007001"
127.0.0.1:6379> HGETALL fraud:features:payer:00000000-0000-0000-0000-000000007002
1) "tx_count_5m"
2) "9"
3) "tx_count_1h"
4) "16"
5) "amount_sum_1h"
6) "3100.00"
7) "reject_count_24h"
8) "2"
9) "distinct_cards_24h"
10) "1"
11) "distinct_devices_24h"
12) "2"
13) "updated_at"
14) "2026-08-05T18:05:00Z"
```

- Ver las características de la tarjeta o del comercio

Para la tarjeta a002:

```
127.0.0.1:6379> HGETALL fraud:features:card:00000000-0000-0000-0000-00000000a002
1) "tx_count_5m"
2) "8"
3) "amount_sum_1h"
4) "3100.00"
5) "distinct_payers_24h"
6) "1"
7) "distinct_devices_24h"
8) "2"
9) "reject_count_24h"
10) "2"
11) "updated_at"
12) "2026-08-05T18:05:00Z"
```

Para el comercio 9002:

```
127.0.0.1:6379> HGETALL fraud:features:collector:00000000-0000-0000-0000-000000009002
1) "tx_count_5m"
2) "12"
3) "amount_sum_1h"
4) "4800.00"
5) "distinct_payers_1h"
6) "4"
7) "reject_count_24h"
8) "3"
9) "reject_rate_24h"
10) "0.25"
11) "updated_at"
12) "2026-08-05T18:05:00Z"
```

- Consultar las relaciones (Sets)

Ver los dispositivos vinculados al pagador 7002:

```
127.0.0.1:6379> SMEMBERS fraud:relations:payer:00000000-0000-0000-0000-000000007002:devices
1) "00000000-0000-0000-0000-000000008002"
2) "00000000-0000-0000-0000-000000008005"
```

- Inspeccionar el stream de eventos fraud:payment_scored

Ver el primer evento registrado en el Stream con todo su payload JSON:

```
127.0.0.1:6379> XRRANGE fraud:payment_scored - + COUNT 1
1) 1) "1786903982450-0"
2) 1) "event_id"
2) "10000000-0000-0000-0000-00000000e003"
3) "event_type"
4) "PaymentScoredEvent"
5) "schema_version"
6) "1"
7) "intento_id"
8) "00000000-0000-0000-0000-00000000b003"
9) "pagador_id"
10) "00000000-0000-0000-0000-000000007002"
11) "cobrador_id"
12) "00000000-0000-0000-0000-000000009002"
13) "tarjeta_id"
14) "00000000-0000-0000-0000-00000000a002"
15) "dispositivo_id"
16) "00000000-0000-0000-0000-000000008002"
17) "monto"
18) "1850.00"
19) "moneda"
20) "ARS"
21) "score_riesgo"
22) "0.9600"
23) "decision"
24) "RECHAZADO"
```

8.2. Consultas tipo CQL definidas para Cassandra son:

- Control de eventos procesados `select_event`

```
cqlsh:fraude> SELECT event_id, processed_at
... FROM eventos_procesados
... LIMIT 10;
```

event_id	processed_at
10000000-0000-0000-0000-00000000e001	2026-08-01 09:05:00.000000+0000
10000000-0000-0000-0000-00000000e003	2026-08-02 11:00:00.000000+0000

(2 rows)

- Log general de decisiones de pago `payment_decision_log_by_day`

Esta tabla concentra toda la información contextual del pago, decisiones y métricas.

```
cqlsh:fraude> SELECT fecha_particion, fecha_hora, event_id, intento_id, pagador_id, cobrador_id, monto, moneda, decision, score_riesgo
... FROM payment_decision_log_by_day
... LIMIT 10;
```

fecha_particion	fecha_hora	event_id	intento_id	pagador_id	cobrador_id	monto	moneda	decision	score_riesgo
2026-08-03	2026-08-03 14:15:00.000000+0000	10000000-0000-0000-0000-00000000e003	00000000-0000-0000-0000-00000000b003	00000000-0000-0000-0000-000000007003	00000000-0000-0000-0000-000000009003	2200.00	USD	RECHAZADO	0.8900
2026-08-02	2026-08-02 11:00:00.000000+0000	10000000-0000-0000-0000-00000000e003	00000000-0000-0000-0000-00000000b003	00000000-0000-0000-0000-000000007002	00000000-0000-0000-0000-000000009002	1850.00	ARS	RECHAZADO	0.9600
2026-08-01	2026-08-01 09:05:00.000000+0000	10000000-0000-0000-0000-00000000e001	00000000-0000-0000-0000-00000000b001	00000000-0000-0000-0000-000000007001	00000000-0000-0000-0000-000000009001	50.00	ARS	APROBADO	0.9600

(3 rows)

- Intentos de pago por pagador `intentos_pago_por_pagador`

```
cqlsh:fraude> SELECT pagador_id, fecha_hora, intento_id, cobrador_id, monto, moneda, decision, score_riesgo
... FROM intentos_pago_por_pagador
... LIMIT 10;
```

pagador_id	moneda	decision	score_riesgo	fecha_hora	intento_id	cobrador_id	mont
00000000-0000-0000-0000-000000007001	ARS	APROBADO	0.0800	2026-08-01 09:05:00.000000+0000	00000000-0000-0000-0000-00000000b001	00000000-0000-0000-0000-000000007001	50
00000000-0000-0000-0000-000000007002	ARS	RECHAZADO	0.9600	2026-08-02 11:00:00.000000+0000	00000000-0000-0000-0000-00000000b003	00000000-0000-0000-0000-000000007002	1850

(2 rows)

- Intentos de pago por comercio `intentos_pago_por_comercio`

```
cqlsh:fraude> SELECT cobrador_id, fecha_hora, intento_id, pagador_id, monto, moneda, decision, score_riesgo
... FROM intentos_pago_por_comercio
... LIMIT 10;
```

cobrador_id	moneda	decision	score_riesgo	fecha_hora	intento_id	pagador_id	mont
00000000-0000-0000-0000-000000009002	ARS	RECHAZADO	0.9600	2026-08-02 11:00:00.000000+0000	00000000-0000-0000-0000-00000000b003	00000000-0000-0000-0000-000000007002	1850
00000000-0000-0000-0000-000000009001	ARS	APROBADO	0.0800	2026-08-01 09:05:00.000000+0000	00000000-0000-0000-0000-00000000b001	00000000-0000-0000-0000-000000007001	50

(2 rows)

- Auditoría de scoring y reglas `decisiones_scoring`

Consulta del detalle técnico de la evaluación del modelo, incluyendo la latencia y el motivo.

```
cqlsh:fraude> SELECT intento_id, decision_id, fecha_hora, decision, score_riesgo, motivo_principal, latencia_ms, reglas_disparadas
... FROM decisiones_scoring
... LIMIT 10;
```

intento_id	decision_id	fecha_hora	decision	score_riesgo	motivo_principal	latencia_ms	reglas_disparadas
00000000-0000-0000-0000-00000000b003	00000000-0000-0000-0000-00000000c003	2026-08-02 11:00:00.000000+0000	RECHAZADO	0.9600	MONTO_Y_VELOCIDA	18	['RF001_HIGH_AMOUNT', 'RF002_PAYER_VELOCITY']
00000000-0000-0000-0000-00000000b001	00000000-0000-0000-0000-00000000c001	2026-08-01 09:05:00.000000+0000	APROBADO	0.0800	RIESGO_BAJ	15	null

(2 rows)

- Detalle de variables evaluadas `evaluaciones_features`

Muestra los valores puntuales de las variables analizadas en el momento del scoring.

```
cqlsh:fraude> SELECT decision_id, feature_id, nombre, valor, fecha_calculo, ventana_temporal
... FROM evaluaciones_features
... LIMIT 10;
```

decision_id	feature_id	nombre	valor	fecha_calculo	ventana_temporal
00000000-0000-0000-0000-00000000c003	amount_sum_1h	amount_sum_1h	6200.00	2026-08-02 11:00:00.000000+0000	1h
00000000-0000-0000-0000-00000000c003	tx_count_5m	tx_count_5m	8	2026-08-02 11:00:00.000000+0000	5m

(2 rows)

8.3. Consultas SQL sobre PostgreSQL

Las consultas SQL representativas definidas para PostgreSQL se estructuran según su propósito analítico:

- **Consulta 1.** Consolidado diario de operaciones, que calcula métricas agregadas operativas como volumen, tasa de rechazo, score promedio y monto total mediante la lectura de la vista materializada `mv_resumen_diario_scoring`.

```

PS C:\Users\yuran\OneDrive\Documents\Luis\1_MIAE\5_BD_IA\tp_bases_de_datos_ai\tp_integrador> docker compose exec postgresql -U fraude -d fraude
psql (16.15)
Type "help" for help.

fraude=# SELECT
fraude-#     fecha,
fraude-#     total_intentos,
fraude-#     aprobados,
fraude-#     rechazados,
fraude-#     ROUND(100.0 * rechazados / NULLIF(total_intentos, 0), 2) AS tasa_rechazo_pct,
fraude-#     score_promedio,
fraude-#     monto_total
fraude-# FROM mv_resumen_diario_scoring
fraude-# ORDER BY fecha;

```

fecha	total_intentos	aprobados	rechazados	tasa_rechazo_pct	score_promedio	monto_total
2026-08-01	2	2	0	0.00	0.1000	170.00
2026-08-02	3	1	2	66.67	0.7300	3100.00
2026-08-03	3	2	1	33.33	0.4133	2515.00
2026-08-04	2	2	0	0.00	0.1450	545.00
2026-08-05	2	0	2	100.00	0.7750	1580.00

```

(5 rows)

fraude=#

```

- **Consulta 2.** Perfil de pagadores de alto riesgo, que identifica los pagadores con mayor nivel de riesgo mediante agrupaciones relacionales y agregación condicional con la cláusula FILTER.

```

fraude=# SELECT
fraude-#     p.pagador_id,
fraude-#     p.pais,
fraude-#     COUNT(*) AS intentos,
fraude-#     SUM(v.monto) AS monto_total,
fraude-#     ROUND(AVG(v.score_riesgo), 4) AS score_promedio,
fraude-#     COUNT(*) FILTER (WHERE v.decision = 'RECHAZADO') AS rechazos
fraude-# FROM vw_detalle_scoring v
fraude-# JOIN pagador p ON p.pagador_id = v.pagador_id
fraude-# GROUP BY p.pagador_id, p.pais
fraude-# ORDER BY score_promedio DESC, monto_total DESC
fraude-# LIMIT 5;

```

pagador_id	pais	intentos	monto_total	score_promedio	rechazos
00000000-0000-0000-0000-000000007005	AR	2	1580.00	0.7750	2
00000000-0000-0000-0000-000000007002	AR	3	3100.00	0.7300	2
00000000-0000-0000-0000-000000007003	UY	3	2515.00	0.4133	1
00000000-0000-0000-0000-000000007004	AR	2	545.00	0.1450	0
00000000-0000-0000-0000-000000007001	AR	2	170.00	0.1000	0

```

(5 rows)

```

- **Consulta 3.** Cálculo del porcentaje de rechazos efectivamente asociados a su activación para medir la efectividad de reglas de negocio.

```

fraude=# SELECT
fraude-#     rf.codigo_regla,
fraude-#     rf.nombre,
fraude-#     COUNT(*) FILTER (WHERE dr.activada) AS veces_disparada,
fraude-#     COUNT(*) FILTER (
fraude-#         WHERE dr.activada AND ds.decision = 'RECHAZADO'
fraude-#     ) AS rechazos_asociados,
fraude-#     ROUND(
fraude-#         100.0 * COUNT(*) FILTER (
fraude-#             WHERE dr.activada AND ds.decision = 'RECHAZADO'
fraude-#         ) / NULLIF(COUNT(*) FILTER (WHERE dr.activada), 0),
fraude-#         2
fraude-#     ) AS rechazo_asociado_pct
fraude-# FROM regla_fraude rf
fraude-# JOIN decision_regla dr ON dr.regla_version_id = rf.regla_version_id
fraude-# JOIN decision_scoring ds ON ds.decision_id = dr.decision_id
fraude-# GROUP BY rf.codigo_regla, rf.nombre
fraude-# ORDER BY veces_disparada DESC, rf.codigo_regla;

```

codigo_regla	nombre	veces_disparada	rechazos_asociados	rechazo_asociado_pct
RF001	Monto elevado	3	3	100.00
RF002	Velocidad del pagador	3	3	100.00
RF003	Dispositivo compartido	3	3	100.00

(3 rows)

- **Consulta 4.** Correlación entre transacciones aprobadas por el modelo y contracargos confirmados de forma posterior.

```

fraude=# SELECT
fraude-#     v.intento_id,
fraude-#     v.fecha_hora,
fraude-#     v.pagador_id,
fraude-#     v.monto,
fraude-#     v.moneda,
fraude-#     v.score_riesgo,
fraude-#     c.fecha_hora AS fecha_contracargo,
fraude-#     c.motivo
fraude-# FROM vw_detalle_scoring v
fraude-# JOIN contracargo c ON c.intento_id = v.intento_id
fraude-# WHERE v.decision = 'APROBADO'
fraude-# AND c.estado = 'CONFIRMADO'
fraude-# ORDER BY c.fecha_hora DESC;

```

core_riesgo	intento_id	fecha_contracargo	fecha_hora	motivo	pagador_id	monto	moneda	s
00000000-0000-0000-0000-00000000b007	0.2000	2026-08-09 11:00:00	2026-08-03 14:13:00	Cuenta comprometida	00000000-0000-0000-0000-000000007003	240.00	USD	
00000000-0000-0000-0000-00000000b005	0.4200	2026-08-08 10:00:00	2026-08-02 11:06:00	Titular desconoce la compra	00000000-0000-0000-0000-000000007002	300.00	ARS	

(2 rows)

- **Consulta 5.** Detección de redes de fraude a partir de patrones de colusión y anomalías físicas con la búsqueda de dispositivos compartidos por múltiples pagadores distintos mediante la cláusula `HAVING COUNT(DISTINCT ...) > 1`.

```

fraude=# SELECT
fraude-#     d.dispositivo_id,
fraude-#     d.tipo_dispositivo,
fraude-#     d.sistema_operativo,
fraude-#     COUNT(DISTINCT pd.pagador_id) AS pagadores_distintos,
fraude-#     MIN(pd.primera_observacion) AS primera_observacion,
fraude-#     MAX(pd.ultima_observacion) AS ultima_observacion
fraude-# FROM dispositivo d
fraude-# JOIN pagador_dispositivo pd ON pd.dispositivo_id = d.dispositivo_id
fraude-# GROUP BY d.dispositivo_id, d.tipo_dispositivo, d.sistema_operativo
fraude-# HAVING COUNT(DISTINCT pd.pagador_id) > 1
fraude-# ORDER BY pagadores_distintos DESC;

```

dispositivo_id	tipo_dispositivo	sistema_operativo	pagadores_distintos	primera_observacion	ultima_observacion
00000000-0000-0000-0000-000000008005	MOVIL	Android	3	2026-03-15 00:00:00	2026-08-05 00:00:00

(1 row)

- **Consulta 6.** Consolidación del histórico de métricas de desempeño registradas para cada versión del modelo de Machine Learning y dataset de entrenamiento.

```

fraude=# SELECT
fraude-#     ms.nombre AS modelo,
fraude-#     ms.version,
fraude-#     ms.estado,
fraude-#     dv.version AS dataset,
fraude-#     mm.nombre AS metrica,
fraude-#     mm.valor,
fraude-#     mm.segmento,
fraude-#     mm.fecha_calculo
fraude-# FROM modelo_scoring ms
fraude-# JOIN dataset_version dv ON dv.dataset_version_id = ms.dataset_id
fraude-# JOIN metrica_modelo mm ON mm.modelo_version_id = ms.modelo_version_id
fraude-# ORDER BY ms.version, mm.nombre;

```

	modelo	version	estado	dataset	metrica	valor	segmento	fecha_calculo
random_forest_fraude	rf-demo-v1	EN_PRODUCCION	dataset-2026-07	auc_roc	0.94	GLOBAL	2026-08-02 00:00:00	
random_forest_fraude	rf-demo-v1	EN_PRODUCCION	dataset-2026-07	precision	0.91	GLOBAL	2026-08-02 00:00:00	
random_forest_fraude	rf-demo-v1	EN_PRODUCCION	dataset-2026-07	recall	0.87	GLOBAL	2026-08-02 00:00:00	

(3 rows)

- **Consulta 7.** Registro de la trazabilidad completa de las acciones operativas y consultas realizadas por los analistas de riesgo dentro del sistema.

```

fraude=# SELECT
fraude-#     _id
fraude-# ORDER BY ea.fecha_hora DESC;
fraude-#     ar.nombre AS analista,
fraude-#     r.nombre AS rol,
fraude-#     ea.fecha_hora,
fraude-#     ea.accion,
fraude-#     ea.tipo_objeto,
fraude-#     ea.objeto_id,
fraude-#     ea.motivo
fraude-# FROM evento_auditoria ea
fraude-# JOIN analista_riesgo ar ON ar.analista_id = ea.analista_id
fraude-# JOIN rol r ON r.rol_id = ar.rol_id
fraude-# ORDER BY ea.fecha_hora DESC;

```

analista	rol	motivo	fecha_hora	accion	tipo_objeto	objeto_id
Bruno Díaz	Auditor		2026-08-08 10:30:00	CONSULTAR_REPORTE	REPORTE	00000000-0000-0000-0000-000000000001
Ana Torres	Analista de riesgo	Contracargo confirmado	2026-08-08 09:30:00	REVISAR_DECISION	DECISION	00000000-0000-0000-0000-000000000005

(2 rows)

- **Consulta 8.** Comparar las predicciones de score en tiempo real frente a la etiqueta definitiva de fraude registrada en el dataset.

```

fraude=# SELECT
fraude=#     CASE
fraude=#         WHEN ds.score_riesgo >= 0.55 THEN 'PREDICE_FRAUDE'
fraude=#         ELSE 'PREDICE_LEGITIMO'
fraude=#     END AS prediccion,
fraude=#     di.etiqueta_fraude,
fraude=#     COUNT(*) AS casos
fraude=# FROM dataset_item di
fraude=# JOIN decision_scoring ds ON ds.decision_id = di.decision_id
fraude=# GROUP BY prediccion, di.etiqueta_fraude
fraude=# ORDER BY prediccion, di.etiqueta_fraude;

```

prediccion	etiqueta_fraude	casos
PREDICE_FRAUDE	f	5
PREDICE_LEGITIMO	f	5
PREDICE_LEGITIMO	t	2

(3 rows)

11. Propuesta para datos semiestructurados, no estructurados o vectoriales

El análisis de la arquitectura revela la presencia de datos que no se amoldan eficientemente a esquemas relacionales rígidos, tales como eventos en tiempo real, snapshots de variables cuantitativas, listas dinámicas de reglas activadas y trazas de auditoría. A continuación, se detalla la representación de estos componentes mediante tecnologías NoSQL e In-Memory, junto con la evaluación de soluciones vectoriales.

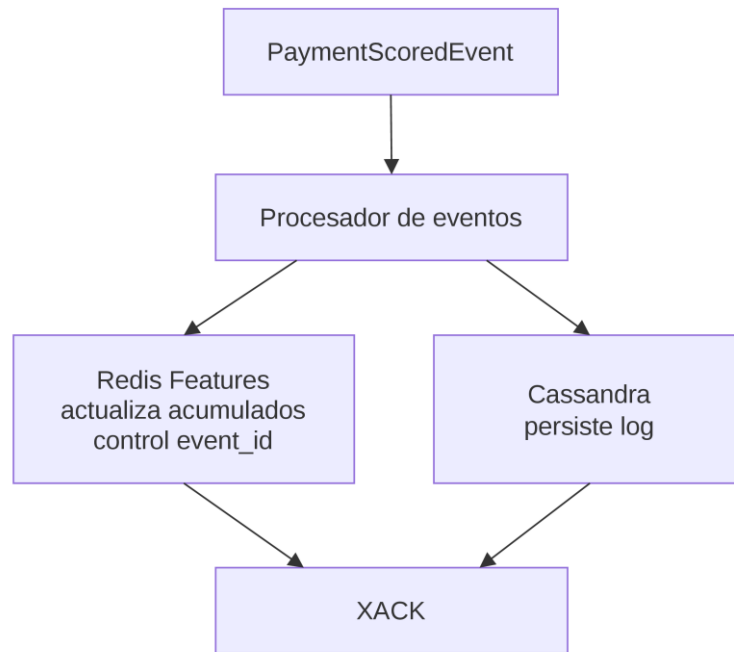
11.1. Identificación y clasificación de datos semiestructurados y eventos

Dentro del ciclo de evaluación y persistencia, se identifican tres tipos de datos con esquemas flexibles o de alta variabilidad:

- A. **Eventos de evaluación en streaming:** representan los mensajes transmitidos de forma asíncrona tras la emisión de un veredicto. Poseen estructuras anidadas que agrupan atributos transaccionales, métricas e información del resultado.
- B. **Atributos cambiantes y métricas acumuladas (snapshots):** representan capturas dinámicas de las variables de riesgo en el instante exacto de la transacción. Su estructura varía en función del tipo de entidad (pagador, tarjeta, dispositivo o comercio) y de la evolución de las reglas de negocio.
- C. **Listas y registros inmutables de auditoría:** constituyen colecciones de elementos variables, como la lista de códigos de reglas de negocio que se dispararon durante una evaluación individual.

11.2. Modelado e Implementación en tecnologías NoSQL e In-memory

Para gestionar esta tipología de datos sin comprometer la latencia del servicio de scoring, se implementa una combinación de estructuras In-Memory (Redis) y un motor NoSQL columnar distribuido (Cassandra).



11.2.1. Redis Features

El componente *Redis Features* organiza los datos de contexto mediante estructuras de tipo *Hash*, optimizadas para lecturas directas por clave con complejidad $O(1)$. Se definen cuatro colecciones clave-valor:

- **PAYER HASH:** clave `fraud:features:payer:{pagador_id}`. Almacena contadores de frecuencia (`tx_count_5m`, `tx_count_1h`), sumas de montos (`amount_sum_1h`), conteo de rechazos (`reject_count_24h`) y diversidad de tarjetas o dispositivos utilizados.
- **CARD HASH:** clave `fraud:features:card:{tarjeta_id}`. Mantiene contadores de transacciones, sumas económicas y métricas de asociación con múltiples pagadores o dispositivos en las últimas 24 horas.
- **DEVICE HASH:** clave `fraud:features:device:{dispositivo_id}`. Registra la frecuencia de uso del dispositivo y el número de identidades o plásticos asociados.
- **COLLECTOR HASH:** clave `fraud:features:collector:{cobrador_id}`. Mantiene métricas de volumen, total de rechazos y tasa de rechazo calculada (`reject_rate_24h`).

A continuación, se representan las estructuras tipo hash en Redis Features:

PAYER_HASH	CARD_HASH	DEVICE_HASH	COLLECTOR_HASH
+key fraud:features:payer:{pagador_id} +integer tx_count_5m +integer tx_count_1h +decimal amount_sum_1h +integer reject_count_24h +integer distinct_cards_24h +integer distinct_devices_24h +datetime updated_at	+key fraud:features:card:{tarjeta_id} +integer tx_count_5m +decimal amount_sum_1h +integer distinct_payers_24h +integer distinct_devices_24h +integer reject_count_24h +datetime updated_at	+key fraud:features:device:{dispositivo_id} +integer tx_count_5m +decimal amount_sum_1h +integer distinct_payers_24h +integer distinct_cards_24h +integer reject_count_24h +datetime updated_at	+key fraud:features:collector:{cobrador_id} +integer tx_count_5m +decimal amount_sum_1h +integer distinct_payers_1h +integer reject_count_24h +decimal reject_rate_24h +datetime updated_at

Para ver el diagrama con mejor resolución puede ir al siguiente enlace:

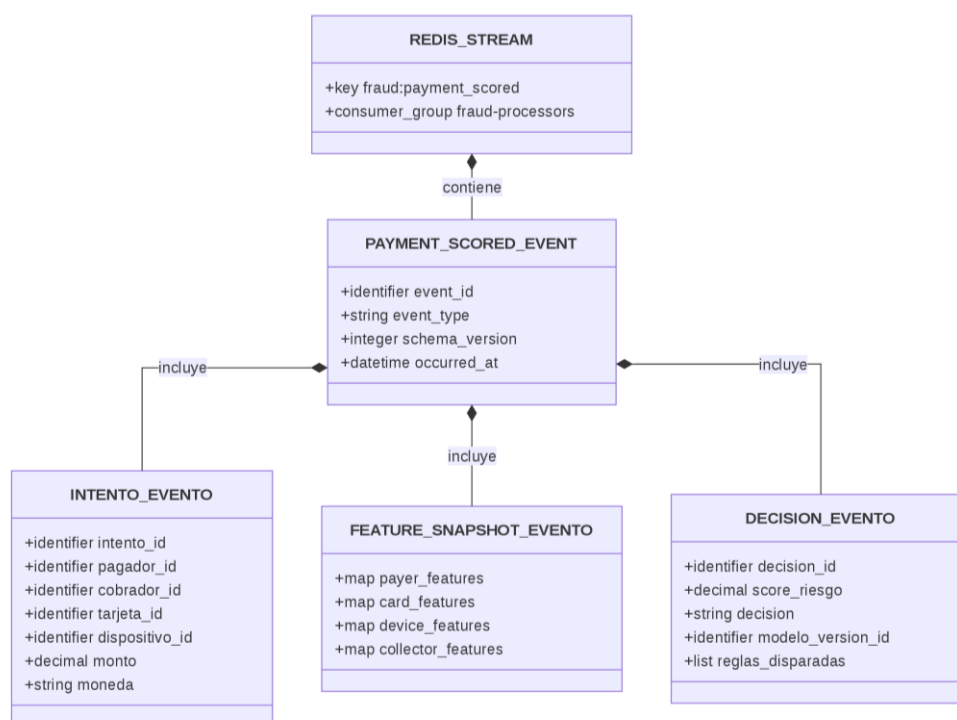
https://github.com/jpskobalski/tp_bases_de_datos_ai/blob/main/tp_integrador/docs/Diagrama%20Redis_Features.drawio.pdf

11.2.2. Redis Streams

El desacoplamiento entre el scoring síncrono y la persistencia asíncrona se resuelve mediante el stream con clave fraud:payment_scored y el grupo de consumidores fraud-processors. Este canal transporta el objeto PaymentScoredEvent, compuesto por los siguientes payloads semiestructurados:

- **INTENTO_EVENTO:** subconjunto de datos con identificadores y montos (intento_id, pagador_id, cobrador_id, tarjeta_id, dispositivo_id, monto, moneda).
- **FEATURE_SNAPSHOT_EVENTO:** mapa estructurado con las métricas consultadas (payer_features, card_features, device_features, collector_features).
- **DECISION_EVENTO:** resultado emitido que incluye decision_id, score_riesgo, decision, modelo_version_id y la lista reglas_disparadas.

El componente **Procesador de Eventos** consume este flujo, actualiza las claves en *Redis Features*, escribe la traza en *Cassandra* y envía la señal de confirmación XACK para validar la lectura del mensaje.



11.2.3. Cassandra

La persistencia masiva de eventos se realiza en la tabla distribuida **PAYMENT_DECISION_LOG_BY_DAY**. El esquema aprovecha los tipos de datos complejos de Cassandra para empaquetar la información semiestructurada:

- **Claves de partición:** **fecha_particion** (tipo date) y **bucket** (tipo integer), las cuales distribuyen la carga de escritura entre los nodos del clúster.
- **Claves de ordenamiento (Clustering):** **fecha_hora** (tipo datetime) y **event_id** (tipo identifier), las cuales garantizan el orden cronológico estricto de las evaluaciones.
- **Atributos columnares complejos:** incluye la columna **feature_snapshot** de tipo Map para guardar el estado exacto de las variables, y la columna **reglas_disparadas** de tipo List para almacenar los códigos de las reglas activadas.

El objetivo de Cassandra dentro de la arquitectura políglota no es gestionar el negocio, asignar casos ni administrar usuarios. Su único propósito técnico en esta capa es actuar como un registro

de auditoría e ingesta masiva de las evaluaciones realizadas. El patrón de acceso esperado para esta capa inmutable es, por un lado, escritura para guardar millones de transacciones por segundo a medida que ocurren. Por otro lado, de lectura que busca extraer las evaluaciones de un rango de tiempo específico, por ejemplo, para los procesos ETL batch que alimentan PostgreSQL o los pipelines de entrenamiento de Machine Learning.

Crear más tablas en Cassandra para representar consultas de pagadores, tarjetas o comercios significaría duplicar innecesariamente los datos desnormalizados, cuando esas necesidades ya están resueltas por Redis y PostgreSQL.

A continuación, se representa la estructura de los datos diseñada para Cassandra:

PAYMENT_DECISION_LOG_BY_DAY
+date fecha_particion PARTITION_KEY
+integer bucket PARTITION_KEY
+datetime fecha_hora CLUSTERING_KEY
+identifier event_id CLUSTERING_KEY
+identifier intento_id
+identifier decision_id
+identifier pagador_id
+identifier cobrador_id
+identifier tarjeta_id
+identifier dispositivo_id
+decimal monto
+string moneda
+string direccion_ip
+string pais_origen
+map feature_snapshot
+decimal score_riesgo
+string decision
+identifier modelo_version_id
+list reglas_disparadas

12. Propuesta de arquitectura de datos

A continuación se expone la propuesta de arquitectura general de datos para la solución de evaluación de riesgo de fraude, el reentrenamiento de modelos de inteligencia artificial y la auditoría del sistema.

12.1. Justificación del enfoque de arquitectura

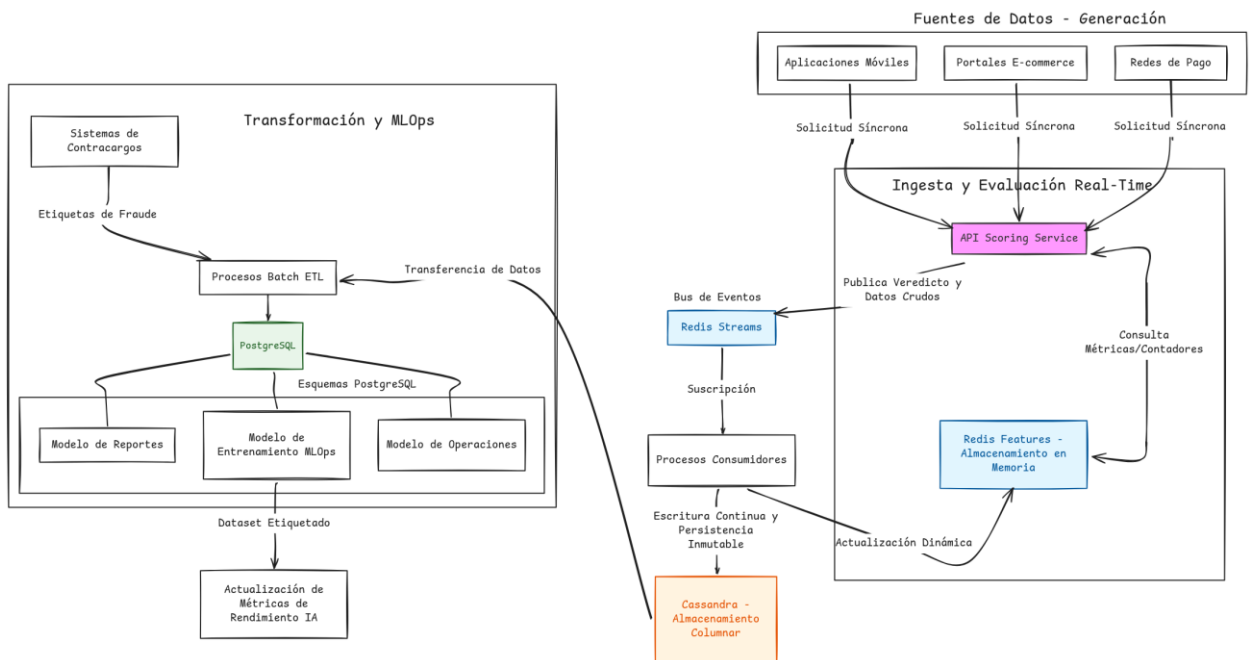
La naturaleza del dominio de detección de fraude en transacciones electrónicas requiere procesar eventos con latencias muy pequeñas para la respuesta en tiempo real. En paralelo, exige registrar volúmenes amplios de información para fines de auditoría y para la construcción de conjuntos de entrenamiento de aprendizaje automático. Una arquitectura simple sustentada en un único motor de base de datos relacional resulta inviable para este escenario. Un motor relacional convencional sufre degradación de rendimiento ante la carga simultánea de cálculo de variables acumuladas en memoria y la ingesta masiva de registros. Por otra parte, un almacén de datos tradicional (Data Warehouse) centralizado presenta latencias de ingesta incompatibles con la respuesta inmediata que exige el punto de venta.

El caso de uso demanda una arquitectura por capas basada en el principio de persistencia polígota y en el procesamiento desacoplado de eventos. Este enfoque distribuye la responsabilidad del almacenamiento y cómputo en tres capas especializadas: una capa operacional en memoria para la lectura de variables de contexto, un motor no relacional columnar distribuido para la ingesta de registros inmutables y un motor relacional estructurado para la gestión transaccional, el control de reglas, el MLOps y la auditoría. Esta división por capas evita cuellos de botella en la base de datos operacional y asegura la consistencia de la información en cada etapa del ciclo de vida del dato.

12.2. Flujo y capas de la arquitectura de datos

El ciclo de vida de la información comprende desde la generación de la solicitud transaccional hasta la actualización de las métricas de rendimiento de los modelos de inteligencia artificial. La circulación de la información se organiza a través de las siguientes etapas:

1. Las fuentes de datos corresponden a las aplicaciones móviles, portales de comercio electrónico y redes de pago que originan las solicitudes de transacción. Estas fuentes envían solicitudes síncronas hacia el servicio de evaluación de riesgo.
2. La ingesta de datos opera mediante un esquema híbrido. En la fase síncrona, la interfaz de programación de aplicaciones (API) del servicio de scoring recibe la solicitud con los datos crudos del intento de pago. Para completar la evaluación en tiempo real, el servicio consulta la capa de almacenamiento operacional en memoria constituida por Redis Features. Esta estructura contiene los datos procesados en forma de métricas acumuladas y contadores dinámicos. Una vez emitido el veredicto, el servicio publica un mensaje en un bus de eventos en streaming sobre Redis Streams. Este mecanismo asíncrono desacopla la respuesta transaccional de la persistencia física en disco.
3. Un grupo de procesos consumidores consume los mensajes del bus de eventos para actualizar las estructuras operacionales y alimentar la capa analítica. Los datos crudos del intento de pago, la captura instantánea de las variables consultadas y la lista de reglas activadas se escriben de forma continua en la capa de almacenamiento analítico inmutable gestionada por Cassandra. Esta tabla distribuida opera bajo un esquema columnar optimizado para la escritura continua de registros históricos sin contención por bloqueos de fila.
4. De forma paralela, procesos de integración y transformación por lotes transfieren la información desde la capa columnar y desde los sistemas de reporte de contracargos hacia la base de datos relacional PostgreSQL. En este componente se estructuran tres esquemas diferenciados: el modelo de operaciones, el modelo de entrenamiento para MLOps y el modelo de reportes. En el esquema de MLOps se estructuran los datos preparados para la inteligencia artificial. Estos conjuntos de datos integran la información histórica de la transacción con la etiqueta definitiva de fraude obtenida tras el periodo de maduración de las disputas bancarias.



12.3. Componentes de consulta, consumidores y usuarios del sistema

La arquitectura habilita mecanismos de acceso adaptados a los perfiles de consumo del sistema, en cuatro componentes principales:

1. El servicio de evaluación de riesgo es el principal consumidor automático de la capa en memoria. Este componente ejecuta las reglas de negocio y los modelos predictivos para emitir una respuesta en milisegundos hacia el punto de venta.
2. Los pipelines de aprendizaje automático consumen los datos preparados en el esquema de MLOps de PostgreSQL para la extracción de características, el reentrenamiento, la validación y el despliegue de nuevas versiones de modelos de inteligencia artificial.
3. Los analistas de riesgo y auditores acceden a la información a través del modelo relacional de operaciones y reportes en PostgreSQL. Este acceso se realiza mediante aplicaciones de gestión y herramientas de consulta estructurada para la administración de reglas de negocio, la revisión de casos sospechosos y el control de auditoría de cambios en el sistema.
4. Los sistemas externos de reporte y tableros de inteligencia de negocios consumen vistas agregadas en el esquema de reportes para el análisis de indicadores clave de rendimiento, tasas de aprobación y volumen de disputas económicas.

13. Estrategia de seguridad, permisos y aislamiento

La arquitectura de datos integra mecanismos de protección diseñados para garantizar la confidencialidad, la integridad y la disponibilidad de la información en todos los componentes del sistema.

13.1. Control de acceso basado en roles y permisos

El control de acceso al sistema se fundamenta en un esquema de control de acceso basado en roles (RBAC), el cual restringe las operaciones de lectura, escritura y administración según las funciones operativas de cada perfil. Los permisos se estructuran mediante la siguiente asignación de responsabilidades:

El perfil de analista de fraude posee acceso de lectura y actualización sobre los casos de revisión asignados en el modelo relacional de operaciones en PostgreSQL. Este usuario consulta evidencias transaccionales, métricas históricas de entidades y adjudica veredictos operativos. No cuenta con permisos para modificar reglas de negocio, umbrales de evaluación, parámetros de modelos de aprendizaje automático ni datos de usuarios del sistema.

El perfil de supervisor de riesgo hereda los permisos de consulta del analista de fraude. Adicionalmente, cuenta con permisos de escritura para la reapertura de casos, la autorización de transacciones que superan umbrales críticos de monto, la asignación de carga de trabajo entre analistas y la propuesta de calibración de reglas. Carece de permisos directos para la promoción de modelos de inteligencia artificial a entornos productivos y para la administración de seguridad del sistema.

El perfil de auditor posee permisos de lectura exclusiva sobre los registros de auditoría almacenados en PostgreSQL y sobre las trazas inmutables registradas en Cassandra. Este usuario examina el historial de acciones, modificaciones de configuración y decisiones operativas. Se le restringe completamente la capacidad de creación, modificación o eliminación de datos, así como el acceso a atributos identificadores directos o datos sensibles de clientes.

El perfil de administrador de reglas cuenta con permisos de creación, modificación y desactivación de condiciones de negocio en el esquema de operaciones de PostgreSQL, además de la gestión de listas de bloqueo y el ajuste de puntos de corte del servicio de evaluación. No posee acceso a la cola de revisión manual de casos, a datos de tarjetahabientes ni a la gestión de cuentas de usuario.

El perfil de administrador de sistema posee permisos globales sobre el catálogo de usuarios, roles y asignación de credenciales. Administra los parámetros de la infraestructura, los conectores externos y el registro de modelos para la promoción o reversión de versiones de inteligencia artificial. No tiene acceso a la gestión operativa de casos ni a la edición de reglas de riesgo.

13.2. Gestión de datos sensibles y tokenización

El tratamiento de datos personales y financieros cumple con los estándares normativos de seguridad de datos de la industria de tarjetas de pago (PCI-DSS) y regulaciones de protección de datos personales.

Los atributos de identificación crítica, tales como el número de tarjeta primaria (PAN), se someten a un proceso de tokenización irreversible antes de su ingesta en el sistema. El token sustituye el valor original en todas las capas de almacenamiento (Redis Features, Cassandra y PostgreSQL). Los datos de identificación de personas y la huella digital de los dispositivos se almacenan mediante funciones de resumen criptográfico (hashing) con salado individual.

El acceso a datos sensibles no tokenizados se bloquea en la capa de persistencia mediante permisos a nivel de columna en PostgreSQL y vistas restringidas. Ningún usuario humano o

aplicación analítica accede al número completo de la tarjeta o a credenciales puras del usuario pagador.

13.3. Restricciones de acceso y aislamiento de inquilinos

La arquitectura implementa un modelo de aislamiento para garantizar la separación de la información entre diferentes comercios o entidades cliente.

El aislamiento en la base de datos relacional PostgreSQL se ejecuta mediante políticas de seguridad a nivel de fila (Row Level Security - RLS). Toda consulta efectuada por un usuario o servicio incluye automáticamente el identificador de la entidad cobradora. El motor de la base de datos filtra las filas visibles en tiempo de ejecución, lo cual impide que un usuario de un comercio acceda a registros transaccionales o métricas de otro cliente.

En la capa operacional en memoria (Redis Features), las claves se estructuran mediante esquemas de espacios de nombres que incorporan el identificador del comercio. El servicio de consulta restringe la lectura a los prefijos autorizados para la entidad solicitante. En el almacenamiento columnar de Cassandra, las claves de partición incluyen el identificador de la entidad, lo cual asegura el aislamiento físico de la información a nivel de almacenamiento distribuido.

13.4. Auditoría y trazabilidad del sistema

El sistema registra de forma automática toda interacción, cambio de estado y evento de consulta sobre la plataforma de datos.

Las modificaciones en las reglas de negocio, ajustes en los umbrales de decisión, promociones de modelos de inteligencia artificial y cambios en los roles de usuario se escriben de manera síncrona en el modelo de reportes y auditoría de PostgreSQL. Este registro almacena el identificador del usuario, la dirección IP de origen, la marca temporal UTC, la acción ejecutada, el valor previo y el valor posterior.

La trazabilidad de la evaluación automática de riesgo se garantiza mediante la ingesta inmutable en Cassandra. Cada decisión de scoring registra el identificador de la solicitud, la captura instantánea de las variables utilizadas, el conjunto de reglas activadas y el resultado emitido. El carácter inmutable de Cassandra impide la alteración o borrado de las trazas de auditoría por parte de usuarios operativos o administrativos.

13.5. Prevención de exposición indebida en componentes de inteligencia artificial

Los pipelines de entrenamiento e inferencia de modelos de inteligencia artificial consumen exclusivamente los datos preparados del esquema de MLOps en PostgreSQL. Estos conjuntos de datos se someten a procesos previos de desanonimización inversa y eliminación de variables de identificación directa. Los modelos operan únicamente sobre variables numéricas, contadores agregados, indicadores categóricos y representaciones vectoriales anonimizadas.

En el esquema propuesto no se realiza la integración con asistentes virtuales o herramientas de recuperación aumentada por generación (RAG) para la consulta de documentación o auditoría en lenguaje natural.

14. Consideraciones de escalabilidad y rendimiento

La arquitectura de datos se diseña para escalar de manera horizontal y mantener la latencia del servicio de evaluación en unos cuantos milisegundos, con independencia del incremento en el volumen transaccional o en el número de usuarios.

La tasa de crecimiento de la información varía según el componente de la persistencia políglota. El almacenamiento analítico inmutable en Cassandra registra el mayor volumen mediante las tablas *PAYMENT_DECISION_LOG_BY_DAY*, intentos por pagador, intentos por comercio, evaluaciones de reglas y evaluaciones de *features*. El modelo relacional en PostgreSQL acumula registros de forma continua a través de los intentos de pago, las decisiones emitidas, las reglas evaluadas y los elementos de *datasets* para entrenamiento. En el almacenamiento en memoria sobre Redis, el crecimiento se vincula al flujo en *Redis Streams* y al volumen de entidades activas guardadas en los *hashes* de contexto.

La eficiencia del sistema depende de tres ejecuciones críticas. La primera es la lectura síncrona en sub-milisegundos de los datos de contexto en Redis por parte del servicio de *scoring*. La segunda corresponde al procesamiento asíncrono de eventos transmitidos en *Redis Streams*, los cuales consume el procesador de eventos para actualizar métricas en memoria y guardar registros inmutables en Cassandra. La tercera consiste en las extracciones por lotes en PostgreSQL para la construcción de conjuntos de datos etiquetados en la capa de *MLOps*.

La gestión del almacenamiento combina esquemas de particionamiento y políticas de retención. La tabla *PAYMENT_DECISION_LOG_BY_DAY* en Cassandra utiliza una clave de partición compuesta por fecha y un *bucket* de balanceo. Las tablas físicas de consulta en Cassandra distribuyen sus datos mediante claves basadas en el identificador del pagador, del comercio o de la decisión. Por su parte, Redis gestiona la memoria a través de tiempos de expiración que eliminan las sesiones a las 24 horas, las ventanas temporales a los 30 días y las métricas acumuladas a los 90 días.

Para evitar cálculos en el punto de venta, el procesador de eventos calcula de forma asíncrona las métricas dinámicas inmediatamente después de cada transacción. Estas variables se actualizan en los *hashes* de Redis e incluyen contadores de operaciones, sumas de montos, tasas de rechazo y conteos de entidades distintas.

El servicio de evaluación síncrona opera desacoplado de la persistencia analítica mediante el bus de eventos en *Redis Streams* y el procesador de eventos. Esta separación aísla al motor en tiempo real frente a los tiempos de escritura en disco de Cassandra y PostgreSQL.

La arquitectura adopta compromisos entre simplicidad, rendimiento y consistencia. La incorporación de tres motores de almacenamiento diferentes incrementa la complejidad operativa y los costos de infraestructura a cambio de garantizar las latencias requeridas. Asimismo, la solución prioriza la disponibilidad en el punto de venta mediante un modelo de consistencia eventual entre la capa en memoria y la persistencia en disco.

14.1. Índices secundarios

La arquitectura de datos propuesta omite el uso de índices secundarios explícitos, esta decisión de basa en los patrones de acceso y en las características estructurales de cada motor seleccionado.

En la capa de almacenamiento en memoria sobre Redis, el acceso a las variables de contexto se realiza mediante operaciones sobre claves directas. Este esquema elimina la necesidad de índices secundarios en memoria.

En la capa analítica inmutable sobre Cassandra, el diseño se rige estrictamente por el principio Query-First, donde la distribución y el ordenamiento de los datos se resuelven mediante el uso de claves de partición para la localización de nodos y claves de clúster para el ordenamiento interno. Evitar los índices secundarios en Cassandra resulta una práctica recomendada en sistemas de alta ingesta, dado que dichos índices generan cuellos de botella al requerir lecturas coordinadas en múltiples nodos del clúster.

En la capa relacional sobre PostgreSQL, las claves primarias y claves foráneas generan de forma automática índices B-Tree subyacentes. Debido a que los patrones de consulta de operaciones, MLOps y reportes se ejecutan a través de cruces por identificadores únicos y búsquedas en rangos temporales clave, la indización implícita de las claves satisface las necesidades de rendimiento sin requerir índices adicionales.

Sin embargo, existe un riesgo de escalamiento y una acción de mejora a futuro, ya que si la capa de reportes y auditoría en PostgreSQL ejecuta búsquedas complejas filtrando por atributos que no forman parte de la clave primaria, la ausencia de índices secundarios obligará al motor a realizar escaneos completos de tabla, esto degradará el rendimiento a medida que aumente el volumen de la tabla.

15. Conclusiones

El desarrollo del presente trabajo práctico permitió abordar de manera integral el diseño y la implementación de un sistema de detección de fraude basado en inteligencia artificial, aplicando los conceptos fundamentales de bases de datos para entornos de alta exigencia operativa. A partir del caso de uso seleccionado, se logró construir una solución que articula tres motores de persistencia: Redis, Cassandra y PostgreSQL; todo en una arquitectura políglota que satisface las demandas contrapuestas de latencia ultrabaja, ingesta masiva de datos y consistencia transaccional.

Uno de los principales aprendizajes obtenidos durante el proceso radica en la necesidad de desacoplar los flujos síncronos y asíncronos del sistema. Esta separación evita que las tareas de escritura en disco o de actualización de acumulados afecten la experiencia del usuario en el punto de venta.

El uso de Cassandra como almacén de logs inmutables se prevé adecuado para soportar el volumen de transacciones que una plataforma de estas características exige, debido a su arquitectura distribuida y su modelo de escritura continua sin contención por bloqueos. La tabla `PAYMENT_DECISION_LOG_BY_DAY`, diseñada con claves de partición compuestas por fecha y bucket, demostró ser efectiva para distribuir la carga de escritura entre los nodos del

clúster y para facilitar las extracciones periódicas necesarias para los procesos batch de entrenamiento y reportes.

Por su parte, PostgreSQL cumplió el rol de motor transaccional primario para la gestión de datos estructurados que requieren integridad referencial y consistencia ACID. Los tres esquemas lógicos definidos: operaciones, MLOps y reportes; permitieron organizar de manera clara las responsabilidades del sistema. Por un lado, la gestión de reglas de negocio y casos de revisión, la preparación de datasets etiquetados para el entrenamiento de modelos de inteligencia artificial, y la generación de reportes operativos y de auditoría. La adopción de la tercera forma normal en estas tablas garantizó la ausencia de redundancia y la trazabilidad de los cambios a lo largo del tiempo.

El ciclo de reentrenamiento continuo de los modelos de machine learning, basado en la retroalimentación de las decisiones humanas y la integración de contracargos confirmados, constituye uno de los pilares de la solución propuesta. Este enfoque de MLOps asegura que el sistema se mantenga actualizado frente a la evolución de los patrones de fraude, y que cada versión de modelo sea debidamente registrada y trazable, permitiendo conocer en todo momento qué versión evaluó cada transacción.

El trabajo permitió verificar la viabilidad de la arquitectura propuesta a través de una implementación mínima en contenedores Docker, que desplegó los tres motores y ejecutó una suite de consultas representativas sobre datos sintéticos. Las pruebas realizadas confirmaron que el flujo de datos funciona de manera integrada, y que las consultas definidas para cada motor cumplen con los requisitos de auditoría, monitoreo y entrenamiento de modelos.

Se identificó que el principal desafío de la arquitectura políglota radica en la complejidad operativa que introduce la coordinación entre tres motores de almacenamiento diferentes, así como en la necesidad de gestionar la consistencia eventual entre la capa en memoria y la persistencia en disco. Sin embargo, las ventajas en términos de rendimiento, escalabilidad y especialización funcional justifican ampliamente esta decisión de diseño para el dominio de detección de fraude, donde la combinación de latencias críticas y volúmenes masivos de datos hace inviable una solución basada en un único motor de base de datos.

Trabajos futuros

La solución desarrollada constituye una base sólida sobre la cual se pueden implementar mejoras y ampliaciones para su evolución hacia un entorno productivo. A continuación, se enumeran las principales líneas de trabajo futuro:

- Migración de la infraestructura a un entorno de producción, con gestión de secretos mediante un servicio externo y eliminación de credenciales del repositorio.
- Incorporación de cifrado en tránsito y en reposo para identificadores, tokens e IP.
- Desarrollo e implementación completa del control de acceso basado en roles (RBAC) para analistas, supervisores, auditores y administradores.
- Establecimiento de mecanismos de reintentos para mensajes fallidos, gestión de la lista de pendientes (PEL) y monitoreo del lag del stream.
- Implementación de procesos de reconciliación periódica entre las capas de Redis, Cassandra y el proceso batch hacia PostgreSQL, para garantizar la consistencia de los datos a lo largo del tiempo.

- Introducción de versionado semántico para contratos de eventos, modelos de inteligencia artificial, reglas de negocio y conjuntos de datos de entrenamiento.
- Implementación de políticas de retención basadas en particionado temporal, y ejecución de pruebas de carga para validar los umbrales de escalabilidad del sistema.